

Grado Universitario en Ingeniería Informática

2024-2025

Trabajo Fin de Grado

“Framework para la evaluación y
optimización de estrategias de
clasificación en problemas de datos
desbalanceados”

Guillermo Villar Sánchez

Tutor

Francisco Javier González Serrano

Leganés, septiembre de 2025



Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento - No Comercial - Sin Obra Derivada**

RESUMEN

Los sistemas de clasificación supervisada suelen enfrentarse a importantes limitaciones cuando los datos de entrenamiento presentan un fuerte desbalance entre clases. Esta asimetría en la distribución de los datos compromete la capacidad del modelo para aprender correctamente las características de las clases minoritarias, afectando directamente a su rendimiento en tareas críticas.

Este Trabajo Fin de Grado presenta el desarrollo de un sistema modular orientado a mejorar el aprendizaje y la clasificación en contextos desbalanceados mediante el estudio para cada problema de distintas estrategias como la conmutación de etiquetas (o *label switching*), entre otras técnicas habituales en este tipo de problemas.

El trabajo parte de una solución previa y se centra en su refactorización y mejora aplicando principios de la ingeniería de Software para optimizar su estructura y eficiencia para permitir el mantenimiento y la escalabilidad. Por otro lado, se ha dotado al sistema de una funcionalidad adicional que permite la detección automática del grado de desbalance en un conjunto de datos y sugiere la combinación más eficaz de estrategias clasificatorias para ese caso concreto.

El resultado final es una aplicación funcional que permite a los usuarios estudiar cual es la mejor estrategia para cada *dataset*, así como una mejora en el núcleo de ejecución que reduce el tiempo de cómputo de los experimentos.

Palabras clave: Redes Neuronales, Desbalance de Clases, Clasificación Binaria, Aprendizaje Supervisado, Label Switching.

DEDICATORIA

A mis padres mi hermana y mi madrina por su entrega que hace que no existan palabras con las que ser capaz de expresar mi agradecimiento hacia ellos.

Gracias por el infinito ímpetu que han puesto en mí y mi educación, y por ser los pilares que me han permitido llegar hasta aquí y desarrollar este trabajo.

Por ello y por todo, hoy y siempre, mil veces gracias. Me habéis convertido en la persona más afortunada en habitar la tierra.

Asimismo, quiero agradecer a Francisco Javier por su guía y apoyo a lo largo y ancho del desarrollo de este proyecto, y por acompañarme a través de los muchos problemas enfrentados.

ÍNDICE DE CONTENIDOS

1.	INTRODUCCIÓN	1
1.1	Motivación	1
1.2	Objetivos generales del Trabajo de Fin de Grado.....	2
2.	Análisis del problema	3
2.1	Definiciones Generales	3
2.3	Definición de la base del código inicial.....	5
2.4	Justificación de decisiones iniciales.....	5
3.	Especificación de requisitos del Sistema.....	8
3.1	Historias de usuario.	8
3.2	Casos de uso.....	9
3.3	Requisitos del sistema.....	11
3.3.1	Requisitos funcionales.	12
3.3.2	Requisitos no funcionales.	20
3.4	Matriz de trazabilidad.	25
4.	ARQUITECTURA DE SOFTWARE.....	28
4.1	Visión general de la arquitectura.	28
4.2	La vista: Interfaz de Usuario.....	29
4.3	El modelo: el corazón del framework.....	33
4.4	El controlador: la conexión.....	41
5.	RESULTADOS	43
5.1	Estadísticas.....	43
5.2	Reportes.	50
6.	CONCLUSIONES	59
6.1	Aprendizajes del desarrollo.	59
6.2	Mejoras futuras.	59
7.	MARCO REGULADOR.....	60
8.	CONTEXTO E IMPACTO SOCIOECONOMICO	62
8.1	Presupuesto y fases.	62
8.2	Impacto socioeconómico.	63
	DECLARACIÓN USO IAG.....	66
9.	BIBLIOGRAFÍA.....	70

ÍNDICE DE FIGURAS

Fig 2.1 Diagrama de Venn conceptos de IA.....	3
Fig 4.1 Modelo MVC	28
Fig 4.2 Estructura de archivos	29
Fig 4.3 Diagrama de procesos	30
Fig 4.5 estructura archivos de la carpeta GUI	32
Fig 4.6 ejemplo código de uso de translations.json.....	33
Fig 4.7 Estructura archivos de la carpeta model.....	34
Fig 4.8 extracto de config.yaml	35
Fig 4.9 diagrama de organización de experimentos	36
Fig 4.11 estructura carpeta connector.....	41
Fig 5.1 Interfaz gráfica desarrollada.....	45
Fig 5.2 Experimentos tiempo imagen 1	46
Fig 5.3 Experimentos tiempo imagen 2.....	46
Fig 5.4 Comparación de métricas con modelos variados	48
Fig 5.5 Comparación de matrices de confusión	49
Fig 5.6 Previsualización PDFs – Página 1 reporte	50
Fig 5.7 Reporte clase binaria Diabetes Pima.....	54
Fig 5.8 Reporte multiclase ECOC	58
Fig 8.1 Diagrama de Gantt	63

ÍNDICE TABLAS

Tabla 3.1 – Historias de usuario	9
Tabla 3.2 – Casos de uso	11
Tabla 3.3 – Requisitos funcionales de desbalance	13
Tabla 3.4 – Requisitos funcionales de modelo	13
Tabla 3.5 – Requisitos funcionales de optimización	14
Tabla 3.6 – Requisitos funcionales de gestión de datos	16
Tabla 3.7 – Requisitos funcionales de multiclase.....	17
Tabla 3.8 – Requisitos funcionales de interfaz.....	19
Tabla 3.9 – Requisitos funcionales de resultados.....	20
Tabla 3.10 – Requisitos no funcionales de rendimiento.....	21
Tabla 3.11 – Requisitos no funcionales de usabilidad.....	22
Tabla 3.12 – Requisitos no funcionales de instalación.....	23
Tabla 3.13 – Requisitos no funcionales de mantenibilidad	24
Tabla 3.14 – Requisitos no funcionales de seguridad	24
Tabla 3.14 – Requisitos no funcionales de seguridad	25
Tabla 3.15 – Matriz de trazabilidad entre casos de uso e historias de usuario.....	26
Tabla 3.16 – Matriz de trazabilidad entre casos de uso y requisitos parte 1	26
Tabla 3.17 – Matriz de trazabilidad entre casos de uso y requisitos parte 2	27

1. INTRODUCCIÓN

1.1 Motivación

Los modelos de Redes Neuronales (ANN de ahora en adelante por sus siglas en Inglés *Artificial Neural Network*) avanzan a una velocidad vertiginosa, tanto en su desarrollo técnico como en su aplicación práctica, y junto con este avance se perciben más casos de uso en múltiples ámbitos de actividades humanas. A medida que estos modelos se integran en más contextos, aumentará el grado en el que se utilizan como apoyo o incluso como único argumento en la toma de decisiones en ámbitos críticos.

Muchos de los problemas reales en los que se utilizarán estos modelos clasificatorios presentan un problema común: tienen una distribución de clases desbalanceada por naturaleza. Entre esos problemas encontramos muchos de alto impacto social, como clasificaciones médicas, detección de fraude financiero, detección de SPAM o de fallos en maquinaria o procesos industriales.

En estos casos de uso, el desbalanceo de clases suele aparecer por una de estas dos fuentes. O bien procede de un muestreo muy general, por ejemplo, hablando de datos médicos podemos imaginar que un muestreo general para la detección de melanomas en la piel encontrará muchos más casos negativos que positivos.

La otra fuente común de desbalances es la detección de eventos raros por la propia naturaleza del evento. Manteniéndonos en el ámbito de datos médicos, el equivalente sería la detección de enfermedades raras, o detección de reacciones poco comunes.

Estos problemas hacen que conocer el desbalance, el volumen de datos trabajado y los metadatos de la fuente no nos den toda la imagen necesaria. Si nuestro modelo de detección de lunares vs tumores dice que siempre son lunares, va a acertar (por lo tanto, tener una accuracy) probablemente mayor a un 90%, pero ese modelo no es útil, porque no está adaptado a la realidad de lo que está estudiando, y no habrá aprendido nada sobre tumores, que es el objetivo real del modelo.

Bases de datos como el ISIC 2020 (International Skin Imaging Collaboration) reflejan grandes desbalances de clases [1], y en múltiples estudios se encuentra la dificultad que añade los desbalances observándose el sesgo hacia la clase mayoritaria de los modelos [2].

Por esto, para la toma de decisiones importantes, no nos podemos apoyar solamente en los números, también hay que analizar el coste relativo de un falso positivo contra un falso negativo, qué estamos haciendo con el modelo y un verdadero entendimiento del caso de uso detrás.

Para lidiar con estos problemas, se utilizan distintas técnicas de aprendizaje automático para mejorar las prestaciones, pero esto puede no ser conocido ni evidente para las personas que se enfrentan a estos que pueden incluso desconocer que un modelo clasificatorio les podría resultar de utilidad. A partir de este enfoque, surge la idea de este proyecto.

Si bien es cierto que se puede enfocar hacia problemas particulares (por ejemplo, una pipeline enfocada a datos médicos en la que, habiendo entrenado el modelo previamente, partiendo de una imagen se puede obtener directamente un diagnóstico), existe una gama tan amplia de casos, que finalmente se decide mantener un enfoque generalista.

1.2 Objetivos generales del Trabajo de Fin de Grado

Este Trabajo de Fin de Grado constituye la culminación del proceso formativo en Ingeniería Informática, orientado a demostrar las competencias adquiridas a lo largo de la carrera. El proyecto tiene como objetivo analizar y mejorar un código y modelos de *machine learning* preexistentes, desarrollando una aplicación que incremente su accesibilidad y prestaciones, integrando así los conocimientos teóricos y prácticos en un resultado final de carácter aplicado.

Se toma como base del proyecto unas directrices proporcionadas por un supuesto cliente, a partir de las cuales se definen y concretan unos requisitos del sistema como base sobre la que articular los objetivos técnicos y funcionales del trabajo.

Estas directrices toman principalmente tres vertientes: El supuesto cliente quiere, en primer lugar, organizar y optimizar su código; en segundo lugar, añadir una serie de funciones; y, por último, mejorar la accesibilidad y usabilidad mediante el desarrollo de una interfaz gráfica para el control de este.

Dentro de las funciones a añadir, el principal objetivo es lograr obtener las mejores recomendaciones de hiper-parámetros o bien, ajustar estos modelos iniciales para enfrentar cualquier problema, o al menos, la gama más amplia posible de problemas que sea razonable enfrentar.

De esta idea se desprende que el objetivo principal de la aplicación o interfaz gráfica de control no es el desarrollo de una interfaz moderna o visualmente sofisticada, sino ofrecer a un público no necesariamente técnico un medio sencillo para obtener recomendaciones sobre los ajustes a emplear en sus clasificadores binarios. En este sentido, el esfuerzo no se centra en dotar a la interfaz de un diseño complejo que pudiera suponer una carga adicional a los procesos de *machine learning* ya de por sí intensivos, sino en la generación de un informe final que presente de manera clara y estructurada los resultados obtenidos, aportando valor añadido en términos de accesibilidad y comprensión.

2. ANÁLISIS DEL PROBLEMA

2.1 Definiciones Generales

Para comprender la base de código de este proyecto, es importante primero definir los conceptos con los que estamos lidiando, y definir también algunas de las estrategias utilizadas para lidiar con el desbalance de clases.

Inteligencia Artificial: Es un concepto difícil de definir, pero la Comisión Europea la define como “Sistemas de software (y posiblemente también de hardware) diseñados por humanos que ante un objetivo complejo, actúan en la dimensión física o digital percibiendo su entorno a través de la adquisición e interpretación de datos estructurados o no estructurados, razonando sobre el conocimiento, procesando la información derivada de estos datos y decidiendo las mejores acciones para lograr el objetivo dado. [3]

Machine Learning (ML) : Es una subdisciplina de la inteligencia artificial que consiste en desarrollar algoritmos y modelos estadísticos capaces de aprender de los datos, identificar patrones y mejorar su rendimiento en tareas específicas de manera autónoma, sin ser programados explícitamente para cada tarea. [4]

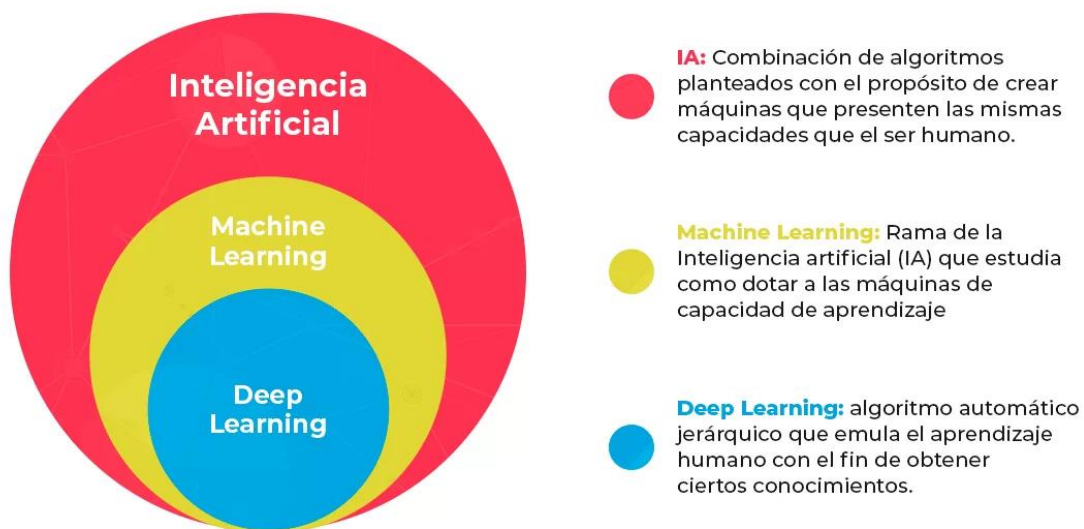


Fig 2.1 Diagrama de Venn conceptos de IA. Obtenida de [5]

Aprendizaje supervisado: El aprendizaje supervisado es una rama del aprendizaje automático en la que un modelo se entrena a partir de un conjunto de datos en el que cada ejemplo de entrada está asociado a una etiqueta o salida deseada. Durante el proceso de entrenamiento, el algoritmo aprende a mapear entradas a salidas basándose en ejemplos previamente etiquetados, de modo que pueda generalizar y predecir correctamente la salida correspondiente para nuevas entradas no vistas. [6]

Clasificación: La clasificación como problema de machine learning supervisado consiste en asignar etiquetas o categorías predefinidas a diferentes casos o muestras de datos. En

este problema, los modelos clasificadores aprenden a identificar patrones en un conjunto de datos de entrenamiento etiquetado (donde cada ejemplo tiene una etiqueta o clase conocida) para luego predecir la etiqueta correcta de nuevas observaciones no clasificadas.

Se da en dos grandes tipos, la clasificación binaria (es decir, problemas en los que hay que diferenciar entre dos casuísticas, una positiva y una negativa), por ejemplo detección de cáncer de mama, o clasificación multiclase, donde un ejemplo clásico de ML es la detección de raza de perro.

Hiper-parámetros: Los hiperparámetros en *machine learning* son variables de configuración que se establecen antes del proceso de entrenamiento y que controlan el comportamiento del modelo o del algoritmo de aprendizaje. A diferencia de los parámetros internos del modelo, que se ajustan automáticamente a partir de los datos durante el entrenamiento (como los pesos en una red neuronal), los hiperparámetros deben definirse de manera explícita por el usuario.

Vamos a lidiar con ellos principalmente en que son el componente que define la “estrategia” que vamos a recomendar para los modelos. Tenemos unos modelos base que necesitan de unos ajustes para funcionar, y la combinación óptima de estos es lo que conforma la estrategia que vamos a recomendar.

2.2 Definiciones de la solución inicial

Desbalance de clases: El desbalance de clases es un problema frecuente en machine learning, especialmente en problemas de clasificación, donde una o más clases tienen muchas menos muestras (clases minoritarias) en comparación con otras que tienen muchas más (clases mayoritarias). Esto significa que la distribución de datos está muy desigual o desequilibrada. Esta situación puede afectar negativamente el rendimiento de los modelos, ya que suelen estar diseñados para trabajar con conjuntos de datos balanceados y pueden tender a predecir siempre la clase mayoritaria, ignorando o clasificando mal las clases minoritarias.

Para lidiar con este desbalance de clases, abordaremos distintas estrategias de machine learning utilizadas en estos casos, y estudiaremos que combinación de estas estrategias es la más aplicable a cada problema en particular que se entregue a la aplicación.

Métodos de remuestreo: Estos métodos modifican los datos de entrenamiento, particularmente enfocándose en el balance de clases de los datos de entrenamiento. Las técnicas como *oversampling* o *undersampling* se basan en cambiar la cantidad de datos en los que se entrena el modelo o bien amplificando la clase minoritaria o reduciendo muestras de la clase minoritaria.

Otra técnica de remuestreo utilizada es el *SMOTE* o *Synthetic Minority Oversampling Technique*. Consiste en generar nuevas muestras de la clase minoritaria combinando datos reales de la misma basado en una interpolación matemática de las muestras minoritarias más cercanas a una real, generando una nueva muestra parecida a las existentes.

Métodos basados en coste: Estas son técnicas que no afectan a la distribución por clases de los datos, sino que adaptan el proceso de aprendizaje para cambiar los costes o

penalizaciones que recibe el modelo al cometer un error en la clasificación, de manera que al aumentar el coste de fallar al clasificar la minoritaria pese a contar con menos muestras podamos hacer que el modelo aprenda sobre la clase minoritaria.

Arquitectura de expertos: La arquitectura de expertos en este proyecto es un ensemble de redes neuronales especializadas donde cada "experto" es una red neuronal entrenada independientemente en diferentes versiones rebalanceadas del dataset.

2.3 Definición de la base del código inicial

La base de código entregada se desarrolla para resolver un problema de clasificación binaria con un elevado desbalance de clases en el contexto de la ciberseguridad. Está orientada al sistema SmartNOC de un Centro de Operaciones de Seguridad (SOC), que procesa y clasifica alertas heterogéneas, con el objetivo de calificar si deberían ser tratadas por agentes expertos o no.

La base del código es muy particular al problema, procesa entradas numéricas y binarias, pero también textuales, entrenando el modelo en palabras relativas a posibles alarmas de ciberseguridad que, detectarían, en su caso, las correspondientes alertas.

Utiliza varias de las técnicas descritas anteriormente, como el *label switching* asimétrico y el *SMOTE* adaptativo, que permiten aumentar la representación efectiva de la clase minoritaria sin comprometer la información de la clase mayoritaria. Estas técnicas se integran dentro de una arquitectura de *ensemble* de expertos (**LSEnsemble**).

Esta arquitectura consiste en un conjunto de 21 o 31 (números impares para evitar empates en la votación) “expertos” que son perceptrones multicapa asimétricos. Esto significa que se entrenan, con datos diferentes, un conjunto de distinto número de la “unidad básica” de IA que es el perceptrón, multicapa porque se entrena una red neuronal de número de capas a determinar por el programa, y asimétrico porque a nivel de la unidad básica o experto del programa podemos dar peso variable a cada clase (enfocado a poder lidiar con grandes desbalances de clase).

Esta base nos es útil a la hora de plantear una solución más generalista, pero también plantea retos a la hora de realizar una transición a problemas generales, como la adaptación del código a problemas multiclase, y la refactorización consecuente al cambiar el problema a tratar.

Este código base entregado por el cliente, nos indica que hemos de hacer un estudio sobre la mejor estrategia de hiper-parámetros de los modelos que usa, de manera que tomaremos este ensemble de expertos como la base sobre la que integrar distintas mejoras y ampliaciones al código. Es importante recalcar en este aspecto, que además de este código de expertos basados en perceptrones, el código inicial del proyecto define otros tipos de expertos, los cuales se podrán seguir utilizando en la aplicación final.

2.4 Justificación de decisiones iniciales

Un aprendizaje fundamental adquirido a lo largo de mi formación es la relevancia que tienen las decisiones iniciales o de carácter arquitectónico en el ámbito de la informática. A menudo, por la propia dinámica de la disciplina, se tiende a priorizar la acción inmediata y comenzar a programar sin una reflexión previa suficiente. Sin embargo, la experiencia demuestra que dichas decisiones iniciales poseen un peso determinante en el desarrollo posterior de cualquier proyecto. Por este motivo, he considerado pertinente dedicar un apartado específico a la justificación de algunas de las elecciones tomadas en las fases iniciales, las cuales condicionarán de manera significativa la evolución del proyecto a lo largo de su ciclo de vida.

El objetivo es acercar las mejores estrategias posibles a un público más general, y esto va a determinar muchas de las decisiones tecnológicas que se van a tomar en este apartado. Como se explicará más adelante, la solución va a ir enfocada a ejecución (tanto de la aplicación como del “backend” en este caso toda la lógica de los modelos) en local.

Esta decisión hace que, para mantener la mayor simplicidad posible, y dado que los usuarios han de instalar Python (manualmente o no), para el desarrollo de la interfaz gráfica utilizaremos Tkinter que se incluye de forma nativa en la mayoría de las instalaciones de Python. Además, es muy ligero, minimalista y se adapta mejor a nuestras necesidades que otras alternativas más pesadas como PyQt o incluso una página web, esta última solución podría encajar mejor en sistemas puramente basados en la nube. Por otro lado, Tkinter nos permite ahorrar recursos físicos que van a ser muy necesarios para los cálculos del modelo, además de aportar simplicidad en código por su compatibilidad multiplataforma nativa.

Los inconvenientes tradicionales de Tkinter para una aplicación de este estilo son principalmente dos: Por una parte, una estética algo anticuada y una ausencia de widgets avanzados o estilos modernos; y por la otra parte, un proceso bloqueante, es decir que si lo ejecutas en el hilo principal junto con otro proceso, la interfaz gráfica dejara de responder.

La forma de lidiar con estos contras es centrar el apartado más “visual” de nuestra aplicación en los reportes de recomendaciones, que se harán utilizando HTML y CSS combinados con ciertas librerías de Python que permiten generar y visualizar archivos PDF desde Tkinter.

Por otra parte, para lidiar con el proceso bloqueante, se diseñará un sistema de *threading* y *polling* que consiste en ejecutar los procesos de ML en hilos de fondo, escribiendo la información que la interfaz gráfica o *GUI* necesita en archivos de progreso que la *GUI* obtiene, mediante *polling* o sondeo, para detectar cambios en estos archivos. Esta decisión sacrifica algo de eficiencia por las llamadas del sondeo, pero mantiene a la *GUI* responsiva y estable, a la vez que se da la opción a usuarios avanzados de poder ejecutar los procesos ML directamente, sin pasar por la interfaz gráfica.

La decisión de la ejecución local obedece, principalmente, a dos factores: el primero, a nivel de seguridad, esta aplicación una vez distribuida (como ejecutable propio o directamente el código) no tendría ninguna comunicación con servidores centrales lo cual, la hace muy segura de cara a garantizar que los datos utilizados quedan únicamente en el ordenador local del usuario.

También la falta de acceso a equipos más avanzados (Las operaciones de ML en las librerías de Python utilizadas mejoran mucho su rendimiento cuando se ejecutan en GPUs con CUDA) han dificultado pensar en otras maneras de entregar la aplicación como podría ser una web que ejecute modelos en la nube y no consuma recursos del usuario. El sistema de compilación para ello de nuevo está desarrollado, pero no había mejoras de cómputo (de hecho, con las herramientas disponibles a las cuentas de alumnos UC3M a través de Google Collab tardaba aproximadamente un 7% más la ejecución de este).

Otra de esas decisiones arquitecturales fundamentales es la de dividir el espacio de búsqueda de hiper-parámetros en dos etapas diferenciadas. Para el espacio de hiper-parámetros usamos la biblioteca `itertools`, en concreto su función `product` que calcula el producto cartesiano de múltiples iterables.

Hay dos grupos de hiper-parámetros que se iterarán en nuestra aplicación. Unos son relativos a la configuración más “clásica” de ML de nuestro modelo, como pueden ser la función de activación, el número de capas ocultas de neuronas del modelo, el número de expertos del “ensemble”, o el número de épocas del experimento. Por otra parte están los hiper-parámetros que tienen más que ver con la configuración para lidiar con el desbalance de clases. Estos son 4, definidos en el código como `Q_RB_C`, `Q_RB_S`, `alpha` y `beta`, que corresponden a los controles del rebalanceo de costes, SMOTE, y cambio de etiquetas de la clase mayoritaria a minoritaria respectivamente.

Esta separación se basa en el principio de optimización jerárquica, mediante el cual podemos asumir, que el mejor modelo general resulta de combinar el mejor modelo de ML tradicional con la mejor estrategia de desbalance. Computacionalmente, esto reduce la complejidad de $O(n_1 \times n_2)$ a $O(n_1 + n_2)$ donde n_1 y n_2 son los espacios de búsqueda de cada etapa. En el nivel más exhaustivo de búsqueda que se implementa en la aplicación, esto reduce las iteraciones de 291,600 experimentos a 1,129, una reducción en el cómputo del 99,6%.

Si bien es cierto que, en algunos casos, se dejan fuera configuraciones (por ejemplo, un modelo ML que por sí solo no fuese el óptimo, pero que podría serlo al combinarlo con alguna estrategia de rebalance), la drástica mejora de rendimiento que acompaña a esta reducción, acompañada de la lógica de encontrar los mejores sub-modelos, hacen que merezca la pena.

3. ESPECIFICACIÓN DE REQUISITOS DEL SISTEMA

Una vez aclaradas las directrices principales de las peticiones del cliente, así como definidos los objetivos del trabajo y, en consecuencia, de la aplicación y el código a desarrollar, resulta pertinente establecer un marco que permita traducir dichas directrices en especificaciones técnicas concretas. Para ello, se presentarán en primer lugar las historias de usuario, que describen de manera narrativa las necesidades desde la perspectiva del usuario final. A continuación, estas se formalizarán en casos de uso que estructuran las interacciones entre el usuario y el sistema. Finalmente, de ambos se derivará un conjunto de requisitos funcionales y no funcionales que guiarán el proceso de diseño e implementación de la solución.

3.1 Historias de usuario.

Las historias de usuario permiten identificar las necesidades y expectativas tanto del cliente que define las directrices iniciales del proyecto como de los usuarios potenciales de la aplicación. Estas narrativas sirven para mantener el diseño orientado al usuario y garantizar que el sistema desarrollado no solo cumpla unos objetivos técnicos, sino que también aporte valor práctico y accesible.

Las historias de usuario “deben ser creadas por el cliente (un PO) y siempre pensando en una finalidad del producto.” [7], por lo que en nuestro contexto habrá dos usuarios principales.

Por una parte, encontramos al cliente que busca una manera más accesible de estudiar y encontrar las mejores prestaciones de su modelo para distintos *datasets*; y por otra parte, investigadores que tengan o no relación con ML, se puedan enfrentar al uso de clasificadores binarios en contextos que requieran de un estudio acerca del desbalance de clases que encuentran.

Se especificarán mediante el formato “*Como [perfil], [quiero] [para].*” [7].

Identificador	Descripción
US1	Como cliente quiero poder elegir el idioma (ES/EN) para que la app que usa mis modelos pueda llegar a mayor público objetivo.
US2	Como cliente, quiero la interfaz cuente con asistente paso a paso y mensajes claros para completar un experimento sin perderme.
US3	Como cliente quiero que la aplicación optimice al máximo los recursos y evite duplicar cálculos para reducir tiempos y costes.
US4	Como cliente quiero que la aplicación use mi modelo LSEnsemble, pero se pueda cambiar el modelo base manteniendo la modularidad.
US5	Como cliente quiero que la aplicación permita todas las etapas de estudio de un <i>dataset</i> multiclase para ganar funcionalidades.

US6	Como cliente quiero que el tratamiento de datos sea ejemplar y cumpla con la normativa en su totalidad para poder distribuirla sin preocupaciones legales.
US7	Como investigador quiero que la app muestre signos del progreso de los experimentos para asegurarme de que se esta ejecutando.
US8	Como investigador quiero poder guardar el progreso que haga en experimentos y se puedan retomar para facilitar los experimentos.
US9	Como investigador no familiarizado con términos de ML quiero poder entender fácilmente los conceptos y controlar el flujo de la app.
US10	Como investigador no familiarizado con términos de ML quiero poder elegir la exhaustividad de los experimentos de manera comprensible.
US11	Como investigador no familiarizado con términos de ML quiero poder descargar informes en los que ver de manera intuitiva los resultados de un experimento.
US12	Como investigador familiarizado con términos de ML quiero sin mucho esfuerzo descubrir si el estudio de un <i>dataset</i> se beneficiará del uso de estrategias específicas para lidiar con el desbalance.
US13	Como investigador médico quiero que la aplicación trate con los datos de forma anónima y cumpla la normativa para poder apoyarme en sus informes y poder tomar mejores decisiones.
US14	Como investigador quiero poder pasar a la aplicación el archivo CSV sobre el que este trabajando sin aplicar cambios para más facilidad de uso de la aplicación.

Tabla 3.1 – Historias de usuario

3.2 Casos de uso.

Si las historias de usuario definen las necesidades de los usuarios y su voluntad de uso, los casos de uso entonces conforman el conjunto de acciones que el usuario toma dentro del sistema para lograr esas voluntades.

El conjunto de casos de uso permite definir las funcionalidades del sistema, la forma de interacción con el usuario y la respuesta que el sistema ha de dar al usuario. Nuestras historias de usuario contemplan dos principales usuarios, el cliente que ordena la aplicación, y los investigadores que serán sus usuarios.

Ambos tienen voluntades distintas para la aplicación, por lo que se consideran historias de usuarios con distinto enfoque, sin embargo interactuarán de la misma manera con la aplicación.

Identificador	Descripción
CU1	Seleccionar idioma (Inglés o Español)
CU2	Seleccionar nivel de familiaridad con términos de ML.
CUS3	Seleccionar fuente de datos sintética (experimentos teóricos)
CU4	Seleccionar archivos desde el sistema operativo del usuario
CU5	Seleccionar fuente de datos estándar de la industria.
CU6	Seleccionar capilaridad o profundidad que tomarán los estudios.
CU7	Seleccionar método de estudio multiclase.
CU8	Aceptar conclusiones de estudio inicial de datos.
CU9	Visualizar estadísticas sobre el <i>dataset</i> seleccionado.
CU10	Visualizar progreso parcial de un experimento ya empezado.
CU11	Borrar datos de un experimento ya empezado.
CU12	Acceder a informe de experimento ya empezado.
CU13	Cargar progreso de un experimento ya empezado.
CU14	Parar desde la aplicación un experimento en progreso.
CU15	Visualizar desde la aplicación el informe generado por un experimento.

CU16	Guardar en el sistema de archivos del usuario el informe generado por un experimento.
CU17	Seleccionar que etapas del entrenamiento incluir en el experimento.
CU18	Poder generar informe añadido sobre prestaciones multiclase.
CU19	Confirmación al usuario del tipo de experimento a ejecutar.

Tabla 3.2 – Casos de uso

3.3 Requisitos del sistema

Una vez definidos los casos de uso, se procede a la especificación de los requisitos del sistema. Estos se organizan en categorías temáticas y se dividen en requisitos funcionales y no funcionales, garantizando así una estructura clara y ordenada. La definición de requisitos se realiza siguiendo las recomendaciones de la norma IEEE Std 830-1998 [8], lo que asegura trazabilidad entre los casos de uso y los requisitos, evita ambigüedades en su redacción y permite asignar una prioridad a cada uno de ellos, facilitando su gestión a lo largo del ciclo de vida del proyecto.

Dividiremos estos requisitos del sistema en dos grandes ámbitos, funcionales y no funcionales, así como una subdivisión por temas. Según el IEEE "Los requisitos funcionales son declaraciones de los servicios que debe proporcionar el sistema, de la manera en que éste debe reaccionar a entradas particulares y de cómo se debe comportar en situaciones particulares" [8].

Estos a su vez los dividiremos en temas aportando una etiqueta individual para cada tema.

- Requisitos de desbalance (RF-DBx)
- Requisitos de modelos (RF-MDx)
- Requisitos de optimización (RF-OPx)
- Requisitos de gestión de datos (RF-GDx)
- Requisitos multiclase (RF-MCx)
- Requisitos de interfaz (RF-UIx)
- Requisitos de resultados (RF-RSx)

De nuevo según el IEEE los requisitos no funcionales son "Un requisito de software que describe no lo que hará el software, sino cómo lo hará el software" [8]. Según esta definición los clasificamos en estos tipos:

- Requisitos de rendimiento (RNF-Rx)

- Requisitos de usabilidad (RNF-Ux)
- Requisitos de instalación (RNF-Ix)
- Requisitos de mantenibilidad (RNF-Mx)
- Requisitos de seguridad (RNF-Sx)
- Requisitos de documentación (RNF-Dx)

Así mismo todos los requisitos seguirán este formato, con la etiqueta temática, prioridad y verificabilidad (alta, media o baja para cada una) y descripción.

RX-Ox	Prioridad: Alta ●	Verificabilidad: Baja ●
Descripción: Esta es una descripción de un requisito de ejemplo.		

3.3.1 Requisitos funcionales.

3.3.1.1 Requisitos de desbalance.

Estos requisitos lidiarán con el conjunto de técnicas y configuraciones de hiperparámetros que el sistema debe implementar para que el modelo LSEnsemble aprenda correctamente y de manera óptima para cada problema como enfrentar *datasets* desbalanceados.

RF-DB1	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe tratar los hiperparámetros específicos para el desbalance de clases (Q_RB_C, Q_RB_S, alpha, beta) como un conjunto de estrategias lógicamente agrupado y configurable, separado de los hiperparámetros de la arquitectura del modelo.		
RF-DB2	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe implementar la técnica de sobremuestreo SMOTE controlada por el parámetro Q_RB_S para generar muestras sintéticas de la clase minoritaria.		
RF-DB3	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe implementar la técnica del rebalance de costes en el nivel de las redes neuronales para cambiar la penalización de errores en la clase minoritaria.		
RF-DB4	Prioridad: Media ●	Verificabilidad: Alta ●
Descripción: El sistema debe añadir a las funciones de pérdida disponibles una que use el factor de rebalance de clases para aumentar la “pérdida” en errores de la clase minoritaria.		

RF-DB5	Prioridad: Media ●	Verificabilidad: Alta ●
Descripción: El sistema debe implementar la técnica de conmutación de etiquetas de dos formas, cambiando las etiquetas de manera aleatoria o cambiando las etiquetas cercanas a la otra clase.		

Tabla 3.3 – Requisitos funcionales de desbalance

3.3.1.2 Requisitos de modelo.

Estos requisitos lidiarán con el conjunto de técnicas y configuraciones de hiperparámetros que el sistema debe implementar para que el modelo LSEnsemble aprenda correctamente y de manera óptima para cada problema como enfrentar *datasets* desbalanceados.

RF-MD1	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El modelo debe en su configuración por defecto utilizar el modelo LSEnsemble, encontrando la mejor combinación de hiperparámetros para este modelo.		
RF-MD2	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe permitir mediante una modificación en el archivo de configuración (config.yaml) dar el paso a otro modelo de los definidos en la carpeta /uc3m recibida como parte del código original.		
RF-MD3	Prioridad: Baja ●	Verificabilidad: Alta ●
Descripción: El sistema debe permitir la exportación del modelo entrenado a través de un objeto serializado de Python <i>pickle</i> con extensión .pkl		

Tabla 3.4 – Requisitos funcionales de modelo

3.3.1.3 Requisitos de optimización.

Estos requisitos funcionales definen el proceso sistemático y automatizado que el sistema debe ejecutar para encontrar la combinación de hiperparámetros que maximice el rendimiento del modelo.

RF-OP1	Prioridad: Alta ●	Verificabilidad: Alta ●
--------	-------------------	-------------------------

El sistema debe realizar una búsqueda automática para encontrar la mejor combinación de hiperparámetros para el modelo activo al accionar el usuario el botón de comenzar experimento.		
RF-OP2	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe implementar una búsqueda jerárquica en dos etapas: una primera etapa (Stage 1) para optimizar la arquitectura del modelo y una segunda etapa (Stage 2) para optimizar las estrategias de desbalance, usando la mejor configuración de la etapa anterior como base.		
RF-OP3	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe permitir al usuario controlar la exhaustividad de la búsqueda de optimización a través de niveles de "capilaridad" predefinidos (1, 2, 3), donde cada nivel corresponde a un espacio de búsqueda de hiperparámetros de diferente tamaño.		
RF-OP4	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe registrar y actualizar de manera iterativa la configuración de hiperparámetros que haya obtenido la mejor puntuación en la métrica de evaluación seleccionada, para presentarla como el resultado final al terminar la búsqueda.		
RF-OP5	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe de generar una etiqueta única a cada dataset que estudie utilizando un algoritmo de <i>hash</i> , para poder registrar el proceso con cada conjunto de datos y así evitar reiniciar los estudios desde cero para cada uno.		
RF-OP6	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe permitir a los usuarios avanzados un “bypass” de la interfaz gráfica completamente para poder dedicar todo el rendimiento de su sistema a los experimentos de ML.		

Tabla 3.5 – Requisitos funcionales de optimización

3.3.1.4 Requisitos de gestión de datos.

Aquí describiremos las capacidades que el sistema debe poseer para la correcta ingesta, tratamiento y gestión interna de los datos de la aplicación.

RF-GD1	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe permitir al usuario seleccionar mediante su sistema de ficheros nativo en formato CSV.		

RF-GD2	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe contar con una funcionalidad para separar automáticamente las columnas de características de aquellas de variable objetivo de los datasets que se cargan mediante archivos externos.		
RF-GD3	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe pasar el formato de un dataset que venga de un archivo externo a el formato de nombres y forma que esperan los modelos.		
RF-GD4	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe aplicar un proceso de normalización o estandarización a las características del dataset para que estén todas en una escala común.		
RF-GD5	Prioridad: Media ●	Verificabilidad: Alta ●
Descripción: El sistema debe ser capaz de leer archivos CSV con diferentes codificaciones de caracteres (ej. UTF-8, Latin-1) para maximizar la compatibilidad con datasets de distintos orígenes.		
RF-GD6	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe analizar los archivos CSV que entregue un usuario para detectar si es un archivo es binario o multiclase.		
RF-GD7	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe dividir automáticamente cualquier dataset cargado en un conjunto de entrenamiento (para entrenar el modelo) y un conjunto de prueba (para evaluarlo), asegurando que la evaluación sea imparcial.		
RF-GD8	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe detectar automáticamente los valores faltantes (nulos) en el dataset y rellenarlos usando una estrategia de imputación.		
RF-GD9	Prioridad: Media ●	Verificabilidad: Alta ●
Descripción: El sistema debe permitir a los usuarios utilizar una funcionalidad de generación de datos sintéticos para experimentos teóricos.		
RF-GD10	Prioridad: Media ●	Verificabilidad: Alta ●

Descripción: El sistema debe permitir una modificación en el archivo de configuración para cambiar las características de la generación sintética de datos.		
RF-GD11	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe eliminar en la propia entrada de archivos del usuario cualquier información personal identificable como DNIs, correos o similar.		
RF-GD12	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe crear y mantener una estructura de directorios organizada para cada dataset y cada ejecución de un experimento, guardando los datos, resultados y logs en sus carpetas correspondientes para garantizar la trazabilidad.		
RF-GD13	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe mezclar los <i>dataset</i> de entrada de manera aleatoria para evitar la aparición de sesgos utilizando una semilla fija definida en el archivo de configuración para que se haga de manera consistente y reproducible.		
RF-GD14	Prioridad: Baja ●	Verificabilidad: Alta ●
Descripción: El sistema debe implementar un pipeline de Procesamiento del Lenguaje Natural (PLN) para transformar automáticamente las características textuales no estructuradas en representaciones vectoriales numéricas.		

Tabla 3.6 – Requisitos funcionales de gestión de datos

3.3.1.5 Requisitos multiclase.

Estos requisitos se centrarán en garantizar que el sistema pueda abordar problemas de clasificación con más de dos clases posibles. Para ello, se definirán las técnicas necesarias para entrenar y evaluar modelos multiclase a partir de clasificadores binarios.

RF-MC1	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe permitir al usuario seleccionar la estrategia de generación de dicotomías entre la OVO (uno contra uno) y OVA (uno contra el resto).		
RF-MC2	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe El sistema debe implementar la lógica interna para descomponer un problema multiclase en múltiples problemas binarios siguiendo la estrategia seleccionada por el usuario.		

RF-MC3	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe entrenar y optimizar un modelo clasificador binario independiente para cada una de las dicotomías generadas por la estrategia seleccionada.		
RF-MC4	Prioridad: Media ●	Verificabilidad: Alta ●
Descripción: Una vez identificada la configuración de hiperparámetros óptima en la fase de optimización inicial, el sistema debe utilizarla para construir el modelo ECOC multiclase completo. Este proceso implica: 1. Instanciar y entrenar un clasificador binario para cada una de las dicotomías definidas por la matriz ECOC (estrategia OvO u OvA), aplicando a todos ellos su configuración de hiperparámetros ganadora. 2. Para la evaluación, agregar las predicciones de todos estos clasificadores individuales sobre el conjunto de prueba. 3. Utilizar la matriz ECOC como un mecanismo de votación o decodificación para traducir el conjunto de predicciones binarias en una única predicción de clase multiclase final. 4. Finalmente, medir el rendimiento del modelo ensamblado comparando las predicciones finales con las etiquetas reales para calcular métricas de rendimiento multiclase.		
RF-MC5	Prioridad: Media ● (Alta si RF-MC4)	Verificabilidad: Alta ●
Descripción: El sistema debe dar la opción al usuario de realizar la construcción del modelo ECOC o simplemente quedarse con el resultado del experimento que ofrece la mejor configuración para cada dicotomía.		

Tabla 3.7 – Requisitos funcionales de multiclase

3.3.1.5 Requisitos de interfaz.

Definen las capacidades y comportamientos de la interfaz gráfica de usuario, el principal punto de interacción entre el usuario y el sistema. Abarcan el conjunto de controles visuales, paneles de información y flujos de interacción que permiten al usuario configurar los parámetros de un experimento, iniciar su ejecución, monitorear su progreso en tiempo real y obtener los resultados finales.

RF-UI1	Prioridad: Media ●	Verificabilidad: Alta ●
Descripción: El sistema debe ofrecer la opción al usuario de seleccionar el idioma (inglés y español), aplicando la elección a todos los componentes de la interfaz, ventanas emergentes e informes.		
RF-UI2	Prioridad: Media ●	Verificabilidad: Alta ●

Descripción: El sistema debe ofrecer la opción al usuario de seleccionar como de familiarizado está con los términos de ML.		
RF-UI3	Prioridad: Media ●	Verificabilidad: Media ●
Descripción: El sistema debe abstraer la complejidad de la optimización de hiper-parámetros a través de un control simple de "Nivel de Capilaridad" (1, 2, 3), permitiendo al usuario elegir entre una búsqueda rápida o una exhaustiva sin necesidad de conocer los parámetros individuales.		
RF-UI4	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: La interfaz debe ofrecer al usuario la opción de elegir entre utilizar un "Dataset Sintético" generado por el sistema, un "Dataset Real" cargado desde el sistema de archivos o un "Dataset clásico de Industria" que demuestra la diversidad de aplicaciones de este sistema.		
RF-UI5	Prioridad: Media ●	Verificabilidad: Alta ●
Descripción: El sistema debe gestionar el contenido textual de la interfaz de manera desacoplada del código fuente. Todas las cadenas de texto visibles para el usuario (etiquetas, botones, mensajes) deben ser cargadas desde un archivo de configuración externo (translations.json). El código debe hacer referencia a estas cadenas mediante un sistema de claves, permitiendo la modificación o adición de idiomas sin necesidad de alterar la lógica de la aplicación.		
RF-UI6	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe calcular y mostrar en la interfaz una estimación del tiempo restante para la finalización del experimento (ETA - Estimated Time of Arrival). Esta estimación debe actualizarse a medida que el experimento avanza.		
RF-UI7	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: Para cumplir con el requisito de responsividad, el sistema debe lanzar el proceso de entrenamiento y optimización en un hilo o proceso separado del hilo principal de la interfaz gráfica, permitiendo que la GUI continúe procesando eventos de usuario de forma independiente.		
RF-UI8	Prioridad: Media ●	Verificabilidad: Alta ●
Descripción: El sistema debe implementar la capacidad de mostrar a los usuarios un "log de ejecución" que registre que botones y modos de uso emplea el usuario, así como mantener un registro de que va ocurriendo durante el uso de la aplicación.		
RF-UI9	Prioridad: Alta ●	Verificabilidad: Alta ●

Descripción: El sistema debe ofrecer a los usuarios avanzados la posibilidad de visualizar desde la interfaz principal los logs avanzados de la propia terminal de ejecución del programa.		
RF-UI10	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: La interfaz debe proporcionar un botón de “Iniciar Experimento” que al ser presionado comience la ejecución del proceso pertinente.		
RF-UI11	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: La interfaz debe ofrecer un botón de "Cancelar" que permita al usuario detener de forma segura un experimento que ya está en ejecución.		
RF-UI12	Prioridad: Media ●	Verificabilidad: Alta ●
Descripción: Una vez iniciado un experimento, la interfaz debe deshabilitar todos los controles de configuración (selección de dataset, capilaridad, etc.) para prevenir cambios que invaliden el proceso en curso.		
RF-UI13	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: La interfaz debe mostrar una barra de progreso y un área de texto que se actualicen periódicamente para informar al usuario sobre el estado del experimento (etapa actual, progreso porcentual).		

Tabla 3.8 – Requisitos funcionales de interfaz

3.3.1.6 Requisitos de resultados.

Estos definen las acciones que debe tomar el sistema para la generación, visualización y guardado de los informes de resultados que se han de obtener al terminar un experimento ML.

RF-RS1	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe ser capaz de generar un informe final en formato PDF, asegurando que el documento sea autocontenido y compatible con cualquier visor de PDF estándar para facilitar su distribución.		
RF-RS2	Prioridad: Media ●	Verificabilidad: Alta ●
Descripción: El informe PDF debe seguir un formato de página estándar (ej. A4) y estar estructurado en secciones claras y definidas, como un documento técnico. Esto incluye un encabezado, un resumen de resultados, detalles de la configuración y gráficos de evaluación.		

RF-RS3	Prioridad: Alta 	Verificabilidad: Alta 
Descripción: La cabecera del informe debe mostrar los metadatos clave del experimento, incluyendo el dataset_id, el timestamp de la ejecución y los parámetros principales (ej. Nivel de Capilaridad, Estrategia Multiclase), para contextualizar los resultados.		
RF-RS4	Prioridad: Alta 	Verificabilidad: Alta 
Descripción: La interfaz debe permitir el visionado del informe PDF en una ventana nueva, así como la posibilidad de guardar el archivo en el sistema de archivos del usuario.		
RF-RS5	Prioridad: Alta 	Verificabilidad: Alta 
Descripción: La interfaz debe dar la opción al usuario de reconstituir los modelos multiclase para obtener métricas del rendimiento tras la recomposición ECOC.		
RF-RS6	Prioridad: Media 	Verificabilidad: Alta 
Descripción: Los resultados de la recomposición ECOC se deben mostrar en un informe siguiendo los requisitos del informe de resultados.		

Tabla 3.9 – Requisitos funcionales de resultados

3.3.2 Requisitos no funcionales.

3.3.2.1 Requisitos de rendimiento.

Estos requisitos definen los estándares de calidad relacionados con la velocidad, la eficiencia y el consumo de recursos del sistema. No describen qué hace la aplicación, sino cómo de bien lo hace en términos de tiempo de ejecución y uso de CPU/memoria, asegurando que la aplicación sea usable pese a la carga de sistema que supone.

RNF-R1	Prioridad: Alta 	Verificabilidad: Alta 
Descripción: El proceso para generar una etiqueta individual para cada archivo, que implica leer y analizar el contenido del CSV, debe completarse en un tiempo razonable para no dar la impresión de que la aplicación se ha bloqueado al cargar un nuevo dataset.		
RNF-R2	Prioridad: Media 	Verificabilidad: Alta 
Descripción: El sistema debe gestionar la memoria de forma eficiente, especialmente al cargar y preprocesar datasets. Los dataframes y objetos intermedios deben liberarse cuando ya no sean necesarios para evitar un consumo de RAM excesivo que pueda ralentizar el sistema del usuario.		

RNF-R3	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema no debe identificar los datasets por su nombre de archivo, sino generando un identificador único basado en sus características intrínsecas (forma, columnas, estadísticas). Esto asegura que si un usuario vuelve a cargar el mismo dataset (incluso con un nombre de archivo diferente), el sistema lo reconocerá y podrá reutilizar los resultados y la estructura ya existentes.		
RNF-R4	Prioridad: Alta ●	Verificabilidad: Baja ●
Descripción: El sistema no consiste únicamente en una interfaz gráfica, sino que se apoya en una serie de scripts y mejoras de código que se moldean alrededor de estos modelos originales. Esos apoyos han de brindar suficientes mejoras como para que la pérdida de cómputo relacionada con la interfaz gráfica nunca supere la ejecución de iteraciones sobre el código original.		
RNF-R5	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe operar como una entidad única, integrada y robusta, no como una colección de scripts independientes. Debe gestionar de forma autónoma todo el ciclo de vida de un experimento, desde la ingesta de datos hasta la generación del informe final. La arquitectura debe garantizar la trazabilidad, reproducibilidad y atomicidad de los resultados, protegiendo al usuario de errores comunes como la sobrescritura de datos o la pérdida de configuraciones.		
RNF-R6	Prioridad: Media ●	Verificabilidad: Alta ●
Descripción: Un error como una excepción en una parte inesperada del código no debe poner en riesgo ni la existencia ni el acceso a los resultados ya obtenidos (aunque sean parciales) de un experimento.		

Tabla 3.10 – Requisitos no funcionales de rendimiento

3.3.2.2 Requisitos de usabilidad.

Estos requisitos definen los estándares de calidad que garantizan que la aplicación sea eficaz, eficiente y satisfactoria de usar. Su objetivo es asegurar una experiencia de usuario positiva, minimizando la carga cognitiva y permitiendo que usuarios con distintos niveles de experiencia puedan alcanzar sus objetivos de forma intuitiva.

RNF-U1	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: La interfaz gráfica debe permanecer siempre responsiva y no congelarse mientras se ejecutan los experimentos. Las acciones del usuario, como mover la ventana o solicitar la cancelación del proceso, deben ser atendidas sin demora. Esto se logra ejecutando el pipeline de entrenamiento en un hilo o proceso separado.		
RNF-U2	Prioridad: Media ●	Verificabilidad: Alta ●

Descripción: El sistema debe abstraer la complejidad de los experimentos, ya sean binarios o multiclase haciendo que sea fácil de configurar, seguir y comprender en cada paso para usuarios no técnicos.		
RNF-U3	Prioridad: Alta ●	Verificabilidad: Alta ●
La interfaz debe guiar al usuario y prevenir activamente la comisión de errores.		
RNF-U4	Prioridad: Alta ●	Verificabilidad: Baja ●
Descripción: El sistema debe adaptarse al conocimiento de ML de cada usuario, simplificando la experiencia para usuarios nóveles, y añadiendo mayores controles a usuarios avanzados.		
RNF-U5	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: La interfaz debe ser consistente en su diseño y comportamiento. Los elementos con funciones similares deben tener una apariencia y ubicación predecibles, y la terminología utilizada en etiquetas y mensajes debe ser coherente en toda la aplicación y en todos los idiomas.		
RNF-U6	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe mantener al usuario constantemente informado sobre su estado, especialmente durante operaciones largas. Esto se logra mediante la visualización en tiempo real de una barra de progreso, el tiempo estimado restante (ETA) y mensajes de estado claros, reduciendo la incertidumbre y la percepción de que la aplicación se ha bloqueado.		
RNF-U7	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe estar diseñado con el tiempo y recursos del usuario en mente, buscando minimizar el tiempo que debe esperar en los experimentos y poniendo todos los recursos disponibles a que sepa y entienda lo que va a tardar y que puede hacer los experimentos en varias partidas de ejecución.		
RNF-U8	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: Para todas las operaciones de entrada (ej. cargar un dataset) y salida (ej. guardar un informe), el sistema debe invocar los diálogos de selección de archivos nativos del sistema operativo del usuario (ej. el Explorador de Archivos en Windows). Esto asegura una experiencia de usuario familiar y consistente.		

Tabla 3.11 – Requisitos no funcionales de usabilidad

3.3.2.3 Requisitos de instalación.

Estos requisitos definen la facilidad con la que un usuario puede instalar y configurar la aplicación en su propio sistema. El objetivo es que el proceso de puesta en marcha sea lo

más simple y automatizado posible, manteniendo la compatibilidad con distintos entornos de sistema operativo.

RNF-I1	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema debe ser compatible con los principales sistemas operativos (Windows, macOS, Linux) sin necesidad de modificaciones en el código.		
RNF-I2	Prioridad: Baja ●	Verificabilidad: Media ●
Descripción: La instalación y ejecución de la aplicación no deben requerir privilegios de administrador. El sistema debe operar exclusivamente dentro de su propio directorio y en los directorios de datos del usuario, sin intentar escribir en ubicaciones protegidas del sistema.		
RNF-I3	Prioridad: Alta ●	Verificabilidad: Alta ●
La aplicación se debe poder descargar con código abierto por parte de usuarios avanzados a través de un repositorio en el que podrán disponer de acceso a todos los archivos del sistema.		
RNF-I4	Prioridad: Alta ●	Verificabilidad: Baja ●
Descripción: El sistema debe adaptarse al conocimiento de ML de cada usuario, simplificando la experiencia para usuarios nóveles, y añadiendo mayores controles a usuarios avanzados.		
RNF-I5	Prioridad: Baja ●	Verificabilidad: Alta ●
Descripción: El sistema se debe poder instalar a partir de un instalador distribuible a los sistemas operativos principales.		

Tabla 3.12 – Requisitos no funcionales de instalación

3.3.2.4 Requisitos de mantenibilidad.

Estos requisitos se centran en la facilidad con la que el código puede ser entendido, modificado, corregido o extendido en el futuro. Un sistema mantenible reduce el coste y el esfuerzo de los desarrollos futuros y la corrección de errores.

RNF-M1	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El código debe seguir las convenciones de estilo del lenguaje (ej. PEP 8 para Python), utilizando nombres de variables y funciones descriptivos y manteniendo una estructura lógica que facilite su lectura y comprensión por parte de otros desarrolladores.		
RNF-M2	Prioridad: Alta ●	Verificabilidad: Media ●

Descripción: Los diferentes módulos del sistema deben estar lo más desacoplados posible. Un cambio en la interfaz gráfica no debería requerir cambios en la lógica de generación de identificadores de dataset, y viceversa.		
RNF-M3	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: Los parámetros que son susceptibles de cambiar (ej. los hiper-parámetros a probar en cada "Nivel de Capilaridad") deben estar definidos en archivos de configuración externos (ej. config.yaml) en lugar de estar "hardcodeados" en el código, facilitando su modificación sin tocar la lógica del programa.		
RNF-M4	Prioridad: Alta ●	Verificabilidad: Baja ●
Descripción: El sistema será desarrollado utilizando un sistema de control de versiones (Git) , permitiendo seguir la estrategia de “ <i>feature branching</i> ” para asegurar la correcta implementación de cada característica de la aplicación y trazabilidad del código.		

Tabla 3.13 – Requisitos no funcionales de mantenibilidad

3.3.2.5 Requisitos de seguridad.

Estos requisitos definen las medidas para proteger el sistema y los datos del usuario contra amenazas, accesos no autorizados o pérdidas de información.

RNF-S1	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema nunca debe modificar los archivos de <i>dataset</i> originales proporcionados por el usuario. Debe trabajar siempre sobre una copia controlada dentro de su propia estructura de directorios.		
RNF-S2	Prioridad: Alta ●	Verificabilidad: Media ●
Descripción: La aplicación local no debe mantener ningún contacto con el resto de archivos del usuario que no seleccione manualmente, ni con ningún servidor ni API centralizado. Tanto la interfaz gráfica como los experimentos funcionarán únicamente en local.		
RNF-S3	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: El sistema no debe almacenar ni registrar información personal identificable (PII) o credenciales. Si se implementara una funcionalidad de anonimización, esta debería ser robusta.		

Tabla 3.14 – Requisitos no funcionales de seguridad

3.3.2.6 Requisitos de Documentación.

Estos requisitos se refieren a la calidad y disponibilidad de la documentación que acompaña al código, tanto para usuarios finales como para desarrolladores.

RNF-D1	Prioridad: Media ●	Verificabilidad: Media ●
Descripción: El código debe incluir comentarios en las secciones que implementen lógica compleja o no obvia, explicando el "porqué" de una decisión de diseño, no solo el "qué" hace el código.		
RNF-D2	Prioridad: Alta ●	Verificabilidad: Media ●
Descripción: El proyecto debe incluir un archivo README.md en su raíz que explique qué es el proyecto, cómo instalarlo, cómo ejecutarlo y un ejemplo básico de uso. Además, habrá otro archivo que explique en concreto como usar la interfaz gráfica.		
RNF-D3	Prioridad: Alta ●	Verificabilidad: Alta ●
Descripción: Cada informe PDF generado debe ser un documento autocontenido que no requiera que el usuario consulte la aplicación para su interpretación. Debe incluir no solo los resultados, sino también el contexto del experimento (parámetros de entrada, dataset_id, timestamp), haciendo que el informe sea comprensible por sí mismo.		

Tabla 3.14 – Requisitos no funcionales de seguridad

3.4 Matriz de trazabilidad.

	US 1	US 2	US 3	US 4	US 5	US 6	US 7	US 8	US 9	US 10	US 11	US 12	US 13	US 14
CU 1	●	●												
CU 2		●		●					●	●				
CU 3									●					
CU 4						●								●
CU 5									●					
CU 6				●	●				●	●				
CU 7					●									
CU 8		●												
CU 9		●												
CU 10							●	●						

CU 11			●			●		●					●	
CU 12			●						●					
CU 13			●											
CU 14								●						
CU 15											●	●	●	
CU 16						●					●			
CU 17			●											
CU 18			●		●									
CU 19		●												

Tabla 3.15 – Matriz de trazabilidad entre casos de uso e historias de usuario

Para mejorar la legibilidad de la matriz de trazabilidad entre casos de uso y requisitos se dividirá la misma en dos partes.

CU1	CU2	CU3	CU4	CU5	CU6	CU7	CU8	CU9	CU10
RF-UI1	RF-UI2	RF-UI4	RF-UI4	RF-UI4	RF-UI3	RF-MC1	RF-UI10	RF-GD6	RF-UI13
RF-UI5	RNF-U4	RF-GD9	RF-GD1		RF-OP3	RF-MC2			RF-UI6
RNF-U5		RF-GD10	RNF-U8		RNF-M3				RNF-U6
			RNF-R1						
			RNF-R3						
			RNF-S1						

Tabla 3.16 – Matriz de trazabilidad entre casos de uso y requisitos parte 1

CU11	CU12	CU13	CU14	CU15	CU16	CU17	CU18	CU19
RF-GD12	RF-RS4	RF-OP5	RF-UI11	RF-RS4	RF-RS4	RF-OP2	RF-RS5	RF-UI10
	RF-RS1	RNF-R3	RF-UI7	RF-RS1	RNF-U8		RF-RS6	
	RNF-D3	RF-GD12	RNF-U1	RNF-D3				

Tabla 3.17 – Matriz de trazabilidad entre casos de uso y requisitos parte 2

Como podemos visualizar en ambas matrices (divididas en tres tablas) se establece una conexión completa y bidireccional a lo largo de todo el ciclo de vida de los requisitos del sistema. Cada historia de usuario, que captura una necesidad o directriz del cliente, se materializa en uno o más casos de uso concretos. A su vez, cada caso de uso está respaldado por un conjunto específico de requisitos funcionales y no funcionales que definen su implementación técnica y sus estándares de calidad.

4. ARQUITECTURA DE SOFTWARE

4.1 Visión general de la arquitectura.

Como se discute en el apartado de justificación de decisiones iniciales, las decisiones arquitecturales tomadas al inicio de un proyecto de software son cruciales para el éxito del mismo.

Se justificó la elección de una ejecución local con Tkinter, la necesidad de un sistema de hilos para garantizar la responsividad de la interfaz y la implementación de una optimización jerárquica para gestionar la complejidad computacional. Este capítulo se adentra en el siguiente nivel de detalle: la arquitectura de software que materializa dichas estrategias en una estructura tangible, organizada y mantenible.

Durante la revisión de la solución original de código nos centramos en qué se hacía, pero no en el cómo. Se partía de estos modelos LSEnsemble, y en un único fichero se gestionaba todo. Desde la entrada de datos, a la iteración y muestra de configuraciones estaba todo en un archivo monolítico de más de 1000 líneas de código.

Este punto de partida, si bien guió los pasos que iba a tomar el proyecto, y como se me mencionó, resultaba muy engorroso de depurar y lidiar con él. Tras tomar esas decisiones iniciales, la decisión de arquitectura parece evidente, ya que voy a contar con una interfaz gráfica, unos procesos ML de fondo y los tengo que conectar o comunicar de alguna manera, por lo que parece ser un MVC (modelo-vista-controlador) claro.

El modelo-vista-controlador se suele utilizar mucho en desarrollo web debido a su facilidad para separar la lógica del “*backend*” con el “*frontend*” sin embargo es un encaje perfecto en nuestra aplicación local debido a las necesidades discutidas y las decisiones tomadas.

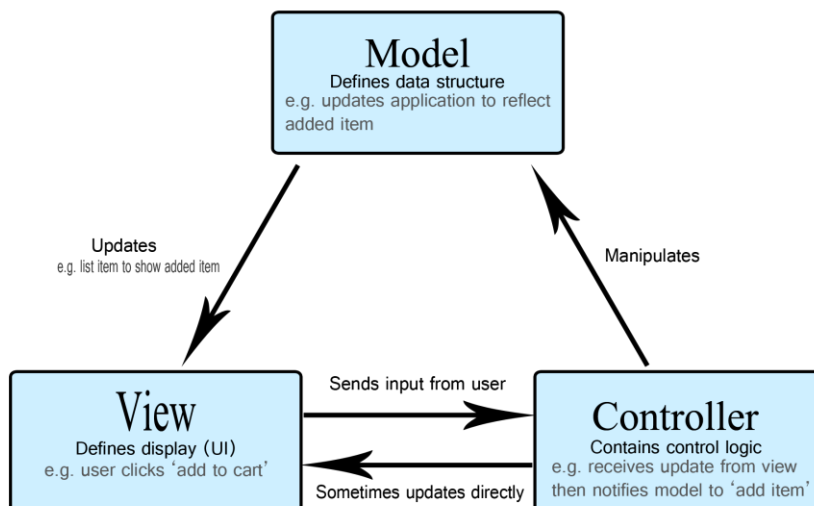


Fig 4.1 Modelo MVC [9]

En nuestro caso, el modelo va a encajar con toda la lógica de iteraciones ML para encontrar la mejor estrategia, pero no se va a quedar únicamente en eso. También será la encargada de la ingesta y trata de datos, de generar los informes y de la lógica multiclas.

La “vista” va a ser la capa visible, como tal al código que gestione la interfaz de Tkinter. Este está encargado sólo del aspecto visual y de registrar la interacción del usuario con la interfaz gráfica.

El controlador es la pieza central en el sentido de que conecta ambas partes. Es el encargado de llevar la información de lo que ocurre en el modelo a la interfaz gráfica para mostrarlo, y a la vez tomar información del usuario a través de la interfaz gráfica e introducirla en el modelo.

Siguiendo esta propia estructura, el código va a quedar organizado en tres carpetas, separadas por cada pieza de la arquitectura, con un archivo externo llamado `main.py` que actuará como punto de entrada o ejecución del código.



Fig 4.2 Estructura de archivos

4.2 La vista: Interfaz de Usuario.

Como ya hemos definido, es el componente con el que interacciona el usuario directamente. Si bien la elección de Tkinter responde bien a nuestras necesidades y nos va a permitir la definición de una interfaz simple que pueda fácilmente cumplir nuestros objetivos, va a sin duda dificultar el desarrollo por ese principal problema de Tkinter, que es la necesidad de operar en el hilo principal.

Esto puede parecer trivial, pero añade varias capas de dificultad al desarrollo de los tres elementos de nuestra aplicación. En una arquitectura MVC tradicional se utilizan eventos de manera que cuando algo ocurre en el modelo se pueda informar a la interfaz de los cambios, en nuestro caso tenía más complicaciones.

El proyecto a desarrollar no es únicamente una interfaz por encima de un modelo no es una pantalla simplemente, sino un controlador completo que va a requerir de mucha información por parte de los procesos ML para poder funcionar. Es por esto que hemos de diseñar un mecanismo de comunicación entre procesos.

Existen múltiples opciones y debemos primero mirar a nuestras necesidades para poder discernirlas. Existen muchas opciones para esto, pero las dos planteadas son mecanismos de comunicación basados en memoria principal (como *queues* o *pipes*) contra archivos, que están basados en memoria secundaria.

Para poder elegir bien tenemos que verdaderamente entender que estamos comunicando y que tratamos de optimizar. Para simplificarlo, vamos a imaginar la tarea de ML como un único proceso y la interfaz como otro único proceso. Al utilizar la librería *multiprocessing* de Python tenemos verdadera ejecución en paralelo de ambos procesos, de manera que el “cuello de botella” o el factor que nos va a hacer perder tiempo es el tiempo que pase el proceso ML bloqueado haciendo una operación de E/S a memoria (sea principal o secundaria).

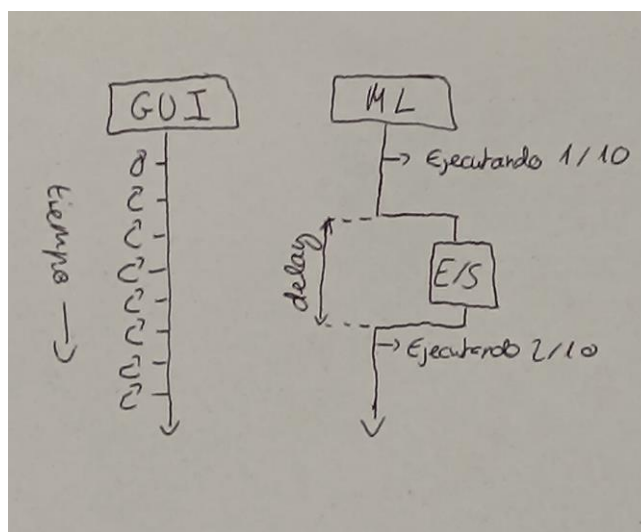


Fig 4.3 Diagrama de procesos

En este diagrama podemos ver que el proceso ML va ejecutando iteraciones y pierde una cantidad de tiempo (en la figura *delay*) que es la cantidad de tiempo que “perdemos” de dedicárselo a iteraciones del proceso ML.

En este sentido la elección parece evidente, minimizar el *delay*. Sin embargo, merece la pena analizar la pérdida de tiempo relativa. Podemos asumir que el método de comunicación que elijamos entre memoria principal o secundaria no va a variar mucho el número de veces que el proceso ha de comunicar, sino que variará el tiempo que tarda cada una.

Como es natural, nuestra aplicación va a lidiar con *datasets* de ML que suelen tener entre cinco y quince mil filas (aunque puede variar drásticamente) lo que puede poner el tiempo total de ejecución de nuestra aplicación (especialmente de archivos multiclase) en el rango de más de 10 horas.

El proceso de ML comunica al terminar una iteración del proceso, estas pueden durar entre dos y cinco minutos. En escala relativa, las dos opciones que tenemos son escrituras a memoria principal que puede tomar de 10 μ s - 1 ms contra escritura a memoria secundaria (100 μ s - 2 ms, dependiendo del tipo de dispositivo, un SSD o disco de estado sólido agiliza mucho estas operaciones). En ordenes relativos entre ambas la memoria principal puede ser hasta veinte veces más rápida, sin embargo son ambas a la vez tan ínfimas en comparación al proceso ML que el tiempo relativo que bloqueamos el proceso es prácticamente insignificante.

En esta revisión académica publicada en la ACM Computing Surveys que estudia el impacto de las operaciones E/S en aplicaciones de ML se argumenta que “En aplicaciones de aprendizaje automático en sistemas HPC, el tiempo dedicado a operaciones de E/S (incluyendo comunicación vía archivos) representa una fracción mínima del tiempo total de ejecución en comparación con las fases de cómputo, especialmente cuando las iteraciones son largas y la comunicación infrecuente.” [\[10\]](#)

Por ello, he decidido implementar la solución que permitía mayor separación de responsabilidades, capacidad de depuración y encaje natural con nuestros requisitos (especialmente gracias a la no-volatilidad del estado), que es la opción de comunicar mediante archivos. Esta nos aporta varias cosas de las que las otras opciones carecen, como, por ejemplo, resiliencia a errores en la comunicación nativa debido a que se guarda en memoria no volátil el progreso de la aplicación constantemente y por tanto, permite guardar el progreso de iteraciones fácilmente.

Se ha decidido utilizar archivos JSON ya que es el formato más ligero disponible para este tipo de operaciones, que se traduce en menos tiempo perdido por el proceso ML pero aun manteniendo legibilidad humana para los usuarios avanzados que quieran revisar los archivos por si mismos.

A su vez, hay que minimizar la cantidad de información escrita en los JSON en general y la información que se actualiza en cada iteración en particular. Para ello, y como el sistema sigue un orden de iteraciones concreto, podemos usar ese orden previamente conocido y tan sólo quedarnos con que configuración ha obtenido la mejor métrica, para compararlo en cada iteración y minimizar de esta forma el espacio utilizado.

Naturalmente la estructura es distinta en el caso multiclase contra el binario, ya que el multiclase contará con un archivo equivalente al “binario” para cada dicotomía y luego un archivo de progreso general que coordine la ejecución de las dicotomías.

Volviendo a la estructura de archivos, se diseñan distintos archivos de código para mantener la división de responsabilidades aún dentro del componente de la interfaz gráfica, ya que de poco serviría diseñar esta arquitectura de software si seguimos construyendo elementos monolíticos por debajo.

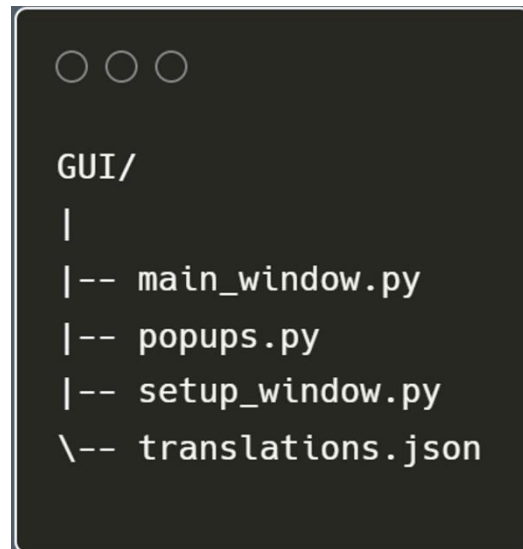


Fig 4.5 estructura archivos de la carpeta GUI

`main_window.py` es el responsable de crear la ventana principal de la aplicación (el objeto `Tk()`) y de definir su layout o estructura de alto nivel. Su trabajo consiste en crear y posicionar los grandes paneles (`ttk.Frame`) que organizan el espacio visual, como el área para la configuración del experimento, la sección para mostrar el progreso y la barra de estado inferior. Actúa como un "ensamblador" que pone los cimientos visuales sobre los que se construirán los demás componentes, pero no contiene la lógica detallada de los eventos.

`popups.py` se encarga de definir la estructura de todos los disclaimers y ventanas que pueden aparecer a lo largo del flujo de ejecución. Desde la información de progreso de un dataset, a las ventanas que informan del estudio inicial de datos, están todos definidos aquí para poder localizarlos con facilidad.

`setup_window.py` es el punto de entrada de la aplicación, se encarga de montar la primera pantalla de elección de idioma y conocimiento del usuario, y mandar esas selecciones al módulo de conector, que es el controlador del modelo.

Finalmente, `translations.json` es un archivo de apoyo muy relevante en esta carpeta. Hay que mantener el código lo más simple y legible posible, y durante el desarrollo, mantenimiento y de cara a usuarios avanzados tiene que ser lo más fácil posible realizar cambios al mismo.

```

○○○

# ARCHIVO -- setup_window.py
# El comando de cada botón llama al método _on_select
tk.Button(button_frame, text="English", command=lambda: self._on_select('en')).pack(side="left", padx=10)
tk.Button(button_frame, text="Español", command=lambda: self._on_select('es')).pack(side="left", padx=10)

#La función on_select guarda la elección del idioma
def _on_select(self, lang_code):
    """Guarda el resultado y cierra la ventana de configuración."""
    self.result = lang_code
    self.destroy()

# ARCHIVO -- main_window.py

# Desde main_window se obtiene el idioma seleccionado
setup = SetupWindow(self.root)
self.language = setup.result
# Para evitar operaciones I/O innecesarias se carga el texto en memoria
def _load_translations(self):
    """Carga el archivo JSON y devuelve el diccionario para el idioma seleccionado."""
    try:
        with open('GUI/translations.json', 'r', encoding='utf-8') as f:
            all_translations = json.load(f)
        # Seleccionamos solo la parte del JSON que corresponde a nuestro idioma
        return all_translations[self.language]

# Todo esto permite que se pueda utilizar la llave del JSON en la creación de elementos:

# En lugar de text="Start Experiment", usamos la llave del diccionario, devuelve el texto en el idioma.
self.start_button = tk.Button(
    self.root,
    text=self.texts.get('start_button_label')
    command=self.start_experiment
)
self.start_button.pack(pady=10)

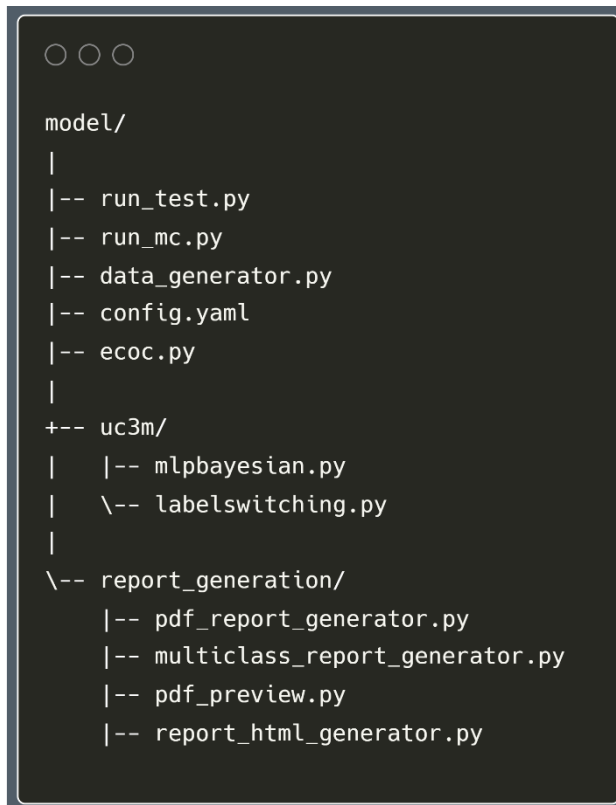
```

Fig 4.6 ejemplo código de uso de translations.json

En este código vemos un esquema del uso de como el uso de un archivo auxiliar añade modularidad, capacidad de mantenimiento y expansión del código, ya que añadir nuevos idiomas o realizar cambios sólo requiere cambios en translations.json, y añadir nuevos botones para esos idiomas.

4.3 El modelo: el corazón del framework.

El modelo contiene todos los elementos necesarios para el cálculo, procesamiento y entrega de los experimentos ML. De nuevo evitamos una arquitectura monolítica especialmente en esta parte, ya que va a ser la más compleja de todas en el código. Esta es la estructura de carpetas seleccionada:



```
model/  
|  
|-- run_test.py  
|-- run_mc.py  
|-- data_generator.py  
|-- config.yaml  
|-- ecoc.py  
|  
+-- uc3m/  
|   |-- mlpbayesian.py  
|   \-- labelswitching.py  
|  
\-- report_generation/  
    |-- pdf_report_generator.py  
    |-- multiclass_report_generator.py  
    |-- pdf_preview.py  
    \-- report_html_generator.py
```

Fig 4.7 Estructura archivos de la carpeta model

La estructura por encima es la siguiente, unos archivos dentro de la carpeta que se encargan de la lógica de la gestión e iteración de los experimentos, así como contar con el archivo de configuración.

Después contamos con la carpeta `uc3m`, que funciona como una librería desde la cual se cargan los modelos (tanto la clase que genera `LSEnsemble` como `MLPBayesian` que es el modelo que compone a cada experto).

Finalmente, contamos con la carpeta `report_generation`, que inicialmente estaba planeada para ser un único módulo, pero entrañaba más dificultad de la esperada. Esto es porque al final van a existir tres tipos de reportes para esta aplicación, el primero de recomendación de estrategias de clasificadores binarios tiene dos opciones (binario o multiclase), y luego va a hacer falta el reporte de recomposición multiclase (se explicará más adelante en que consiste).

Otra razón es que lidiamos con tecnologías distintas. Los reportes se van a generar mediante HTML y CSS combinados, de modo que tiene sentido separarlos ya que no serán de mucho interés a usuarios avanzados (principalmente interesados en el archivo de configuración y sus relaciones).

Esta carpeta utiliza `Jinja2`, una librería de Python que funciona como motor de plantillas, lo que nos permite separar el código HTML de la lógica de Python, para poder introducir en “Python puro” la información a los informes, manteniendo una plantilla de HTML (con CSS embebido) estándar y estable para todos los experimentos que la compartan.

La lógica para ello está dividida en distintos archivos. `Report_html_generator.py` monta la “maqueta” del html, `multiclass_report_generator.py` le pasa la información sobre el tipo de reporte que es, que nombres poner y que datos mostrar.

Después el `pdf_report_generator.py` genera el archivo pdf que el usuario puede guardar, y `pdf_preview_popup.py` prepara la visualización del archivo pasando el pdf a imágenes usando la librería `pdf2image` para que se pueda previsualizar en la GUI de Tkinter, pasándole esas imágenes a la vista por medio del conector.

Sobre el archivo de configuración, va a funcionar digamos que de una manera similar a `translations.json` en cuanto a que va a actuar como referencia externa sobre la que va a leer el cuerpo de los propios experimentos. En lugar de decir bueno, este bucle se iterará 10 veces, en el archivo de configuración quedará definido en un apartado que el código referenciará, de modo que, desde ese archivo, se pueda gestionar el funcionamiento del programa sin tener que acceder a sus archivos más complejos.

Se elige el formato YAML, ya que no va a ser leído tantas veces como los JSON por ejemplo de E/S y el aumento relativo de tamaño se puede ignorar, mientras que nos permite tener estructuras anidadas que al aumentar su complejidad mantienen su legibilidad mejor frente al JSON. Otra gran ventaja que tiene frente al JSON es soporte nativo para comentarios, que nos va a permitir explicar exactamente que controla cada parámetro a usuarios avanzados.

En él, por ejemplo, se definen los niveles de capilaridad que va a utilizar la aplicación. Como ya se ha mencionado antes, están separados entre los más “relativos” al modelo ML base y los más relativos a lidiar con el desbalance.

```
○○○

capilaridad_levels:
  # --- Nivel 1: Búsqueda Rápida (Baseline) ---
  # Diseñado para una validación rápida del pipeline. Utiliza un espacio de búsqueda muy limitado.
  1:
    max_seconds_per_model: 120 # Límite de tiempo (wall-clock) para el entrenamiento de cada configuración individual
    del modelo.
    n_simulations: 1           # Número de repeticiones del experimento con la misma configuración para evaluar la
    estabilidad del modelo.
    # --- Stage 1: Hiperparámetros de la Arquitectura del Experto (Red Neuronal) ---
    stage1:
      activation_fn: relu      # Función de activación para las capas ocultas. Opciones: 'relu', 'sigmoid', 'tanh'.
      drop_out: [0.0]         # Tasa de dropout para la regularización de las capas ocultas. Rango: [0.0, 1.0].
      hidden_size: [30]       # Número de neuronas en la capa oculta de cada experto.
      lbfgs: false            # Booleano para usar el optimizador L-BFGS. Opciones: true (L-BFGS), false (Adam/SGD,
      según implementación).
      loss_fn: custom         # Función de pérdida. Opciones: 'custom' (implementación propia), 'cross entropy', 'mse'.
      mode: random            # Modo de búsqueda de hiperparámetros. Opciones: 'random' (búsqueda aleatoria), 'grid'
      (búsqueda en malla).
      n_batch: [128]          # Tamaño del lote (batch size) para el entrenamiento.
      n_epoch: [50]           # Número máximo de épocas de entrenamiento.
      num_experts: [21]        # Número de modelos (expertos) que componen el ensemble.
      rb_each_expert: false    # Estrategia de rebalanceo. Opciones:
      # false: Se aplica un rebalanceo global al dataset una única vez. Todos los expertos usan
      el mismo dato.
      # true: Cada experto realiza su propio rebalanceo de datos. Mayor coste computacional.
    # --- Stage 2: Hiperparámetros del Ensemble y del Rebalanceo Específico ---
    stage2:
      Q_RB_C: [0.0]           # Factor de coste para el rebalanceo por coste de la clase minoritaria.
      Q_RB_S: [1, 2]          # Factor de sobremuestreo para SMOTE (Synthetic Minority Over-sampling Technique).
      alpha: [0.0, 0.1, 0.2]  # Probabilidad (0 a 1) de aplicar Label Switching de una etiqueta de la clase
      mayoritaria a la minoritaria.
      beta: [0.0]             # Probabilidad (0 a 1) de aplicar Label Switching de una etiqueta de la clase minoritaria
      a la mayoritaria.
```

Fig 4.8 extracto de config.yaml

En esta parte del archivo de configuración podemos ver la explicación exhaustiva de cada apartado, con las opciones disponibles. Podemos observar la definición del nivel de capilaridad más pequeño, que en esta configuración tendría una iteración de stage1 para tomar ese modelo como *baseline*, y 6 de parámetros de lidiar con el desbalance.

Ahora vamos a explicar los archivos que se encuentran en la carpeta model. Se encargan de gestionar lo que en su momento habíamos llamado “proceso ML”, y tiene dos partes principales, una la del tratamiento de datos (gestionados por `csv_preloading.py` y `data_generator.py`) y otra de organización de experimentos (`run_mc.py`, `run_test.py`, y `test_LSEnsemble.py`).

La organización de experimentos es la siguiente:

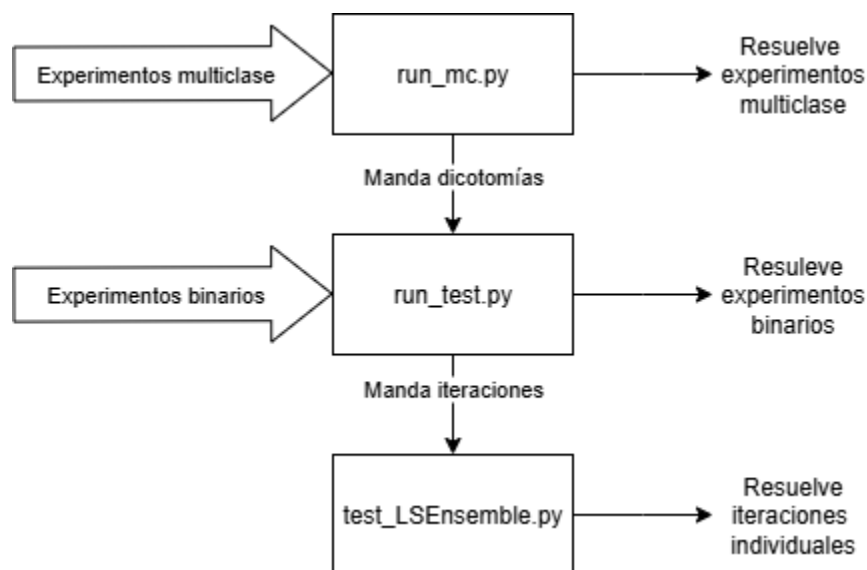


Fig 4.9 diagrama de organización de experimentos

En un alto nivel, nuestra aplicación consiste en resolver cierta cantidad de iteraciones individuales, esto es, un conjunto particular de hiperparámetros que resuelven el modelo sobre un *dataset*. Esta resolución es de lo que se encarga `test_LSEnsemble.py`. Para experimentos binarios, el controlador es `run_test.py`, que se encarga de probar la combinación de valores que toque entrenando en cada iteración un clasificador binario por cada iteración requerida.

Finalmente, el orquestador de experimentos multiclase es `run_mc.py`, que se encarga de coordinar todo lo necesario para la ejecución multiclase, pero antes, debemos explicar qué es un experimento multiclase y cómo se gestionan.

Un experimento multiclase es aquel que en la columna que nuestro programa haya designado como *target* u objetivo tiene más de 2 opciones posibles, pero recordemos que los modelos de los que partimos de base son clasificadores binarios. Estos son sólo capaces de hacer predicciones sobre dos clases en un conjunto de datos, por lo que no podemos directamente predecirlos así.

Una opción fácil sería utilizar modelos (como otros tipos de redes neuronales que tengan múltiples neuronas como capa de salida) que tengan un soporte nativo a predicción multiclase.

Esa modificación iría contra los requisitos definidos ya que el objetivo es encontrar la mejor estrategia para estos modelos, por lo que nos queda una opción, la descomposición en dicotomías.

La idea general es que, si un modelo de clasificación binaria no puede aprender ni predecir un conjunto de datos con múltiples clases, podemos hacer lo siguiente. Generar una serie de dicotomías utilizando distintas estrategias, de manera que cada nuevo sub-problema sea resoluble por nuestros clasificadores binarios. Cada uno de estos sub-problemas se denomina dicotomía. El sistema implementa dos estrategias principales para esta descomposición, definidas en el archivo `config.yaml` y ejecutadas por el módulo `csv_preloading.py`:

One-vs-One (OVO): Esta estrategia consiste en crear un clasificador binario para cada par de clases posible. Por ejemplo, si un dataset tiene tres clases {A, B, C}, OVO generará tres dicotomías y entrenará tres modelos distintos: uno para distinguir A de B, otro para A de C, y un tercero para B de C. El número de clasificadores necesarios es $N * (N-1) / 2$, donde N es el número de clases. Aunque requiere un mayor número de modelos, cada uno se entrena con un subconjunto de datos más pequeño y generalmente más balanceado, lo que puede ser beneficioso.

One-vs-All (OVA) o One-vs-Rest: Esta estrategia es más directa. Se entrena un único clasificador binario por cada clase, enseñándole a distinguir esa clase específica del resto de las clases combinadas. Para el mismo ejemplo de {A, B, C}, OVA generaría tres modelos: el primero para distinguir A de {B, C}, el segundo para B de {A, C}, y el tercero para C de {A, B}. El número de clasificadores es simplemente N. Esta aproximación es computacionalmente más eficiente, pero puede crear problemas de desbalance severo en cada dicotomía (una clase contra muchas).

Tras iterar cada una de las dicotomías como problema particular ya tendremos la mejor estrategia encontrada para entrenar cada clasificador binario, y por lo tanto parece que hemos logrado el objetivo (es de lo que se informará en el primer informe generado).

Sin embargo, el objetivo final no es solo optimizar los clasificadores individuales, sino utilizarlos para resolver el problema multiclase original. Aquí es donde entra en juego la matriz ECOC (Error-Correcting Output Codes), que es el núcleo de la estrategia de recomposición.

La matriz ECOC es una tabla que define cómo se estructura el problema multiclase en un conjunto de problemas binarios. En esta matriz, las **filas representan las clases originales** (ej. 'Gato', 'Perro', 'Pájaro') y las **columnas representan los clasificadores binarios** que hemos entrenado. Cada celda contiene un valor, típicamente +1 o -1, que define a qué "lado" de la dicotomía pertenece cada clase para ese clasificador específico.

Por ejemplo, para un problema con 3 clases {A, B, C} y una estrategia OVO (One-vs-One), tendríamos 3 clasificadores binarios y la matriz ECOC se vería así:

Clase	Clasificador 1 (A vs B)	Clasificador 2 (A vs C)	Clasificador 3 (B vs C)
A	+1	+1	0
B	-1	0	+1
C	0	-1	-1

(Nota: El '0' indica que la clase no participa en esa dicotomía específica).

Tabla 4.1 Matriz ECOC OVO

Clase	Clasificador 1 (A vs B & C)	Clasificador 2 (B vs A & C)	Clasificador 3 (C vs B & A)
A	+1	-1	-1
B	-1	+1	-1
C	-1	-1	+1

(Nota: No hay ceros debido a que todas las clases participan al ser One vs All).

Tabla 4.2 Matriz ECOC OVA

A partir de aquí, una nueva predicción pasa por todos los clasificadores binarios entrenados. Imaginemos que estamos operando con la matriz OVA descrita arriba, y estamos ante una muestra que es de la clase A. El primer clasificador (A vs Rest) predecirá A. En el segundo clasificador, B vs Rest, predecirá Rest, como lo hará con el tercer clasificador.

Esto genera un vector de predicción así: (+1, -1, -1). Ahora se compara con el *codeword* de cada clase (el *codeword* de A está resaltado en amarillo, consiste en qué tendría que predecir el modelo para que se considere de esa clase). Después, mediante el algoritmo de distancia de Hamming (que mide cuantas posiciones son distintas en un codeword) se predice la clase que tenga menor distancia. En este caso:

- Codeword A = (+1, -1, -1) → distancia = 0 (coincide exactamente).
- Codeword B = (-1, +1, -1) → distancia = 2.
- Codeword C = (-1, -1, +1) → distancia = 2.

El caso expuesto es un ejemplo, pero en realidad en el código se ha implementado un intervalo de confianza, y gracias a la diversidad que aporta tener un *ensemble* de expertos que han visto versiones distintas de los datos permite tener distintos puntos de vista y hace prácticamente imposible un verdadero empate en esta consideración.

El análisis de esta cuestión es opcional (ya que la mejor estrategia para cada clasificador ya estaba encontrada) pero fundamental para entender los problemas multiclase, y por eso damos la opción a los usuarios de generar el reporte sobre la reconstitución.

Finalmente, acerca del control de datos existen dos archivos en `model`, ambos con una función bien definida. El primero, `csv_preloading` lidia con los archivos externos que entran en la aplicación, y el segundo, `data_generator` se encarga de lidiar con el sistema de carpetas y la generación de datos sintéticos.

La misión del primer archivo es muy ambiciosa: permitir que cualquier *dataset* pueda ser estudiado por la aplicación, sea como sea su formato interno. Para ello, utiliza una serie de funciones clave.

La primera de ellas: `identify_target_column` busca distintas opciones dentro del dataset para tratar de encontrar la columna objetivo. Para ello busca nombres típicos (`target`, `label`, `objective`, `predicción`, `result`, y...). Además, busca diferencias con el resto del dataset. Si todas las columnas de un dataset son valores continuos y la última o la primera son binarios con valores 0/1 probablemente sea la columna objetivo.

Si no es capaz de identificarla con suficiente confianza, se le pregunta al usuario. Otra de las funciones clave es la de `detect_multiclass`. Para ello también se emplean múltiples métodos, como por ejemplo encontrar una columna con valores discretos entre 0-10 al final de una columna, o encontrar varias columnas que compongan una “one-hot-encoding”. Para desarrollar este método se han estudiado *datasets* estándar de la industria como MNIST o los expuestos en el repositorio UCI y los patrones en los que exponen los archivos multiclase para detectar esos patrones.

Este código también se encarga de dejar los archivos estandarizados, no sólo normalizando la información en ellos, sino también unificando el formato al esperado por la aplicación (columnas de feature numeradas, columna objetivo, llamada `target`, sin información personal identificable...), y guarda tanto el archivo original entregado como un archivo terminado en `_processed` que la aplicación utilizará.

La última función más relevante de este archivo es la de generar las dicotomías necesarias para archivos multiclase una vez el usuario confirma el método a utilizar.

Para poder abordar de la mejor manera posible esta tarea tan ambiciosa se ha introducido una importante restricción. El sistema actual no es capaz de lidiar con datos textuales, dado que estandarizar su tratamiento y poder aprender información relevante para los modelos ML sin restringir temáticamente la aplicación resulto incompatible con las buenas prestaciones frente a archivos generalistas con formatos muy variados.

Con el fin de mantener una estructura de archivos organizada encontramos la función más importante de `data_generator.py`, `generate_dataset_id`. El funcionamiento es el

siguiente:

```
def generate_real_data_id(file_path):
    try:
        df = pd.read_csv(file_path)

        # 1. Metadatos básicos del dataset
        metadata = {
            'shape': df.shape,
            'columns': sorted(df.columns.tolist()),
            'dtypes': {col: str(dtype) for col, dtype in sorted(df.dtypes.items())}
        }

        # 2. Estadísticas rápidas y determinísticas
        stats = {}
        for col in df.columns:
            if df[col].dtype in ['int64', 'float64', 'int32', 'float32']:
                # Para columnas numéricas: media, std, min, max
                stats[col] = {
                    'mean': round(df[col].mean(), 6) if not df[col].isna().all() else None,
                    'std': round(df[col].std(), 6) if not df[col].isna().all() else None,
                    'min': df[col].min() if not df[col].isna().all() else None,
                    'max': df[col].max() if not df[col].isna().all() else None
                }
            else:
                # Para columnas categóricas: número de únicos y muestra de valores
                unique_vals = df[col].dropna().unique()
                stats[col] = {
                    'unique_count': len(unique_vals),
                    'sample_values': sorted([str(v) for v in unique_vals[:5]]) # Primeros 5 valores
                }

        # 3. Combinar todo de manera determinística
        dataset_signature = {
            'metadata': metadata,
            'statistics': stats
        }

        # 4. Generar hash
        signature_str = str(sorted(dataset_signature.items()))
        hash_obj = hashlib.md5(signature_str.encode())
        dataset_hash = hash_obj.hexdigest()[:8]

        return f"real_{dataset_hash}"
```

Fig 4.10 *generate_real_data_id()*

La etiqueta para un programa local como el nuestro se podía derivar con facilidad directamente del path del archivo, por ejemplo, pero se ha decidido hacer analizando por completo las características del dataset, teniendo en cuenta las columnas, filas sus medias, valores máximos y mínimos, valores distintos que hay para asegurarnos que aunque se cambie el nombre o la localización los usuarios no malgastarán recursos repitiendo experimentos ya realizados.

Esto de hecho abre la puerta a una posible funcionalidad a añadir en el futuro. Una opción puede ser permitir a los usuarios subir (de forma verificada) sin necesidad de tener

usuarios ni nada parecido, los resultados de un hash en particular a una base de datos. Esto nos permitiría implementar una especie de “cache” de experimentos, de manera que en la nube tendríamos sólo experimentos y sus resultados, manteniendo así la privacidad total de nuestro sistema, pero permitiendo a los usuarios reducir la carga en sus equipos particulares.

Todo el resto del archivo se encarga de generar los distintos directorios necesarios para experimentos multiclase o de clase binaria, teniendo estos directorios de experimentos reales como ejemplo.

4.4 El controlador: la conexión.

Como ya expuesto, este funciona como el controlador del MVC, llevando los datos del modelo a la interfaz y viceversa.

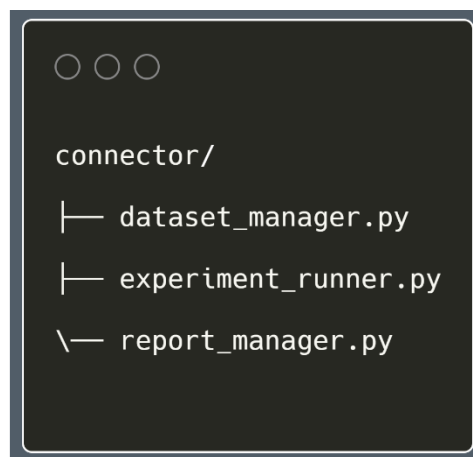


Fig 4.11 estructura carpeta connector

`dataset_manager.py` actúa como el gestor principal del ciclo de vida de los datasets, sirviendo de puente entre la interfaz y la lógica ML del model. Su función no es la de generar los identificadores o definir la estructura de carpetas por sí mismo, sino la de orquestar el proceso llamando a las funciones correspondientes que residen en el model. Por ejemplo, cuando se añade un nuevo fichero de datos, el `dataset_manager.py` invoca a `generate_real_data_id` del model para obtener el identificador único. Una vez que el model le devuelve este ID, el `dataset_manager` lo utiliza para llamar a otra función del model, `get_dataset_directory_structure`, y así obtener las rutas exactas donde debe almacenarse todo.

Con esta información, procede a realizar las operaciones en el sistema de archivos, como crear los directorios y copiar los ficheros. De esta manera, se encarga de comunicar a ambas partes la información necesaria: pide al model los datos lógicos (IDs, rutas) y ejecuta las acciones físicas, informando del resultado. Además, expone funciones que permiten listar los datasets ya existentes, una capacidad que la interfaz gráfica utiliza para poblar los menús de selección.

La función de `experiment runner` es la de orquestar la ejecución de los experimentos de machine learning. Cuando el usuario inicia un experimento desde la interfaz,

el `experiment_runner.py` recibe el identificador del dataset y los parámetros de configuración del modelo. Su primera acción es crear un directorio único para esta ejecución específica, utilizando una marca de tiempo para asegurar la trazabilidad y evitar colisiones. A continuación, en lugar de importar y ejecutar el código del modelo directamente, construye un comando y lanza un “runner” (el binario o multiclase) como un subproceso del sistema operativo. Este enfoque es fundamental, ya que le permite capturar la salida de la consola en tiempo real y retransmitirla a la interfaz, ofreciendo al usuario avanzado un log en vivo del progreso del entrenamiento. Su labor concluye cuando el subproceso termina, momento en el que puede notificar a la interfaz que el experimento ha finalizado.

Finalmente, `report_manager.py` se encarga de la gestión de reportes y visualización de resultados. Su rol de nuevo no es el de generar los informes en sí, sino el de actuar como un intermediario que sabe dónde encontrar los resultados y quién sabe cómo procesarlos. Al recibir una solicitud desde la interfaz para ver un informe (especificando un dataset y una ejecución), el `report_manager.py` localiza la carpeta de la ejecución correspondiente y los ficheros de resultados crudos (por ejemplo, `test_results.csv`). Seguidamente, invoca a los módulos especializados que residen en la subcarpeta `model/report_generation/`, pasándoles las rutas a estos ficheros. Son estos módulos los que realizan el cálculo de métricas y la generación de los informes en formato HTML o PDF. Finalmente, recibe la ruta al informe ya generado y la devuelve a la interfaz para que esta pueda mostrarlo al usuario o permitir su descarga.

5. RESULTADOS

5.1 Estadísticas.

En la fase de pruebas se desarrolló un conjunto amplio de estadísticas e indicadores con el objetivo de verificar distintas etapas del funcionamiento de la optimización de parámetros (desde el preprocesado y el particionado hasta la búsqueda y la evaluación). De entre todas ellas, se seleccionan para el informe aquellas más representativas de los objetivos del TFG, evitando sobrecargar la presentación con métricas redundantes.

Además, no se priorizan los números crudos del “mejor” modelo en términos absolutos, sino las diferencias relativas entre los modelos base y aquellos que incorporan la mejor estrategia para lidiar con un desbalance concreto. Este contraste es el que realmente informa sobre la efectividad de las decisiones de diseño.

Para medir estadísticas utilizamos una estructura de archivos peculiar, en la que cada experimento binario, (puede ser un experimento binario real o una dicotomía) cuenta con un directorio runs, en el que se inicializa una carpeta nueva con un timestamp cada vez que se ejecuta el experimento, y en él se van guardando estadísticas sobre cada iteración.

Evidentemente, se guardan cosas necesarias como los parámetros y la métrica encontrada en cada iteración, pero también se guarda cuánto tiempo tardan, así como otras métricas no seleccionadas por el usuario como la matriz de confusión y una serie de datos que se utilizan en esta etapa. A partir de estas métricas de tiempo, por iteración y tiempo por experimento, se ha estudiado el efecto de las mejoras definidas en el apartado de arquitectura. Al efecto de estudio estadístico se guardan también el número de etiquetas cambiadas de clase mayoritaria a minoritaria y viceversa, mediante el uso de conmutación de etiquetas.

Para la ejecución de las pruebas se ha utilizado un conjunto de *datasets* recogido en el apartado de industria del proyecto. Estos se han seleccionado para ejemplarizar la variedad de campos en los que aparecen este tipo de problemas. Se observan desbalances diversos, algunos son binarios y otros multiclase, en este caso se ha contado como tiempo de ejecución el tiempo medio por dicotomía, ya que el código original no permitía la ejecución de estos de manera iterada.

Las fuentes escogidas para estos datos son Kaggle y el repositorio de la Universidad de California Irvine, ambas fuentes son consideradas reputadas y fiables por la comunidad académica, ya que recopilan *datasets* de referencia que sirven como bancos de pruebas estandarizados para la evaluación de algoritmos con licencia libre abierta.

Son lo siguientes:

- Glass Forensic (6 clases)
Proveniente del UCI Machine Learning Repository, *Glass Identification Data Set*. [\[11\]](#)
- E.coli Protein (8 clases)
Proveniente del UCI Machine Learning Repository, *E. coli Data Set*. [\[12\]](#)

- Breast Cancer
Proveniente del UCI Machine Learning Repository, *Breast Cancer Wisconsin (Diagnostic)*. [[13](#)]
- Diabetes Pima Indians
Proveniente del UCI Machine Learning Repository, *Pima Indians Diabetes Database*. [[14](#)]
- Spam Detection
Proveniente del UCI Machine Learning Repository, *Spambase Dataset*. [[15](#)]
- Customer Churn
Proveniente de Kaggle, *Telco Customer Churn* (IBM sample dataset). [[16](#)]
- Credit Card
Proveniente de Kaggle. Credit Card Fraud Detection Dataset de Kaggle (European Credit Card transactions, 2013). [[17](#)]
- Bank Marketing
Proveniente de UCI. Estudio sobre la suscripción a depósitos sobre llamadas de marketing. [[18](#)]

Los estudios en estos *datasets* podrán ser ejecutados por los usuarios utilizando el menú de pruebas estandarizadas disponibles en la interfaz gráfica. Como no es una práctica profesional, para subir grandes archivos (como es el caso de los conjuntos de datos ML) se ha utilizado la librería `lfs` que permite subir una referencia que se descargará localmente al clonar el repositorio.

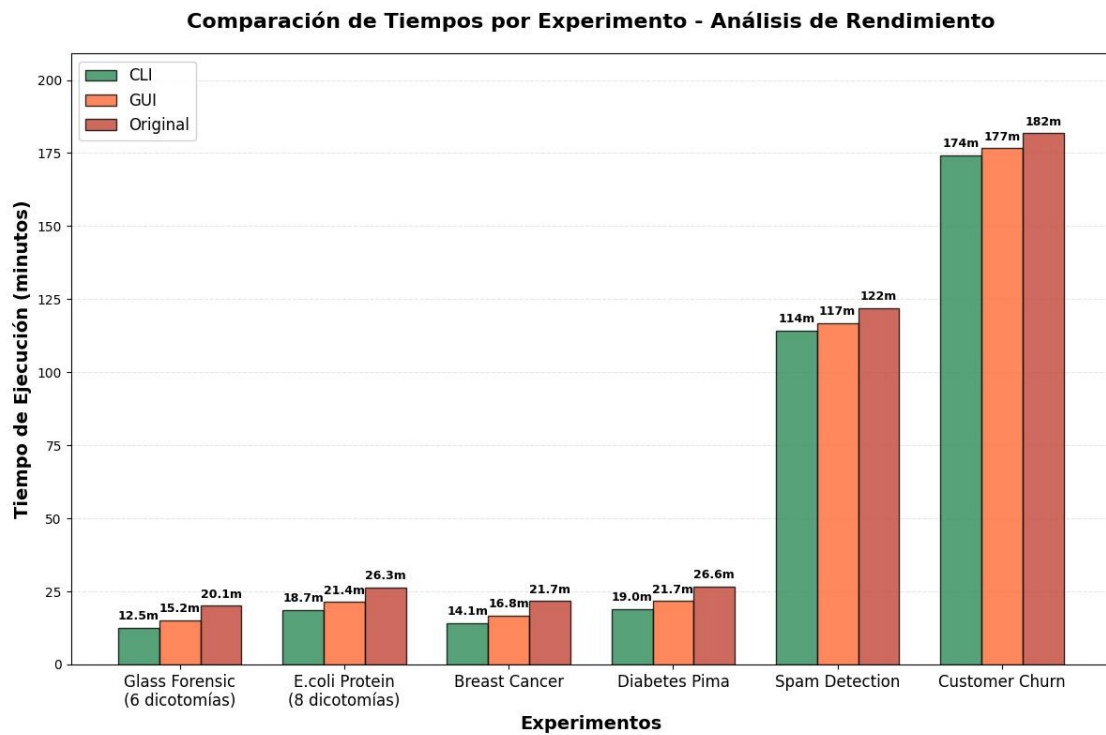


Fig 5.2 Experimentos tiempo imagen 1

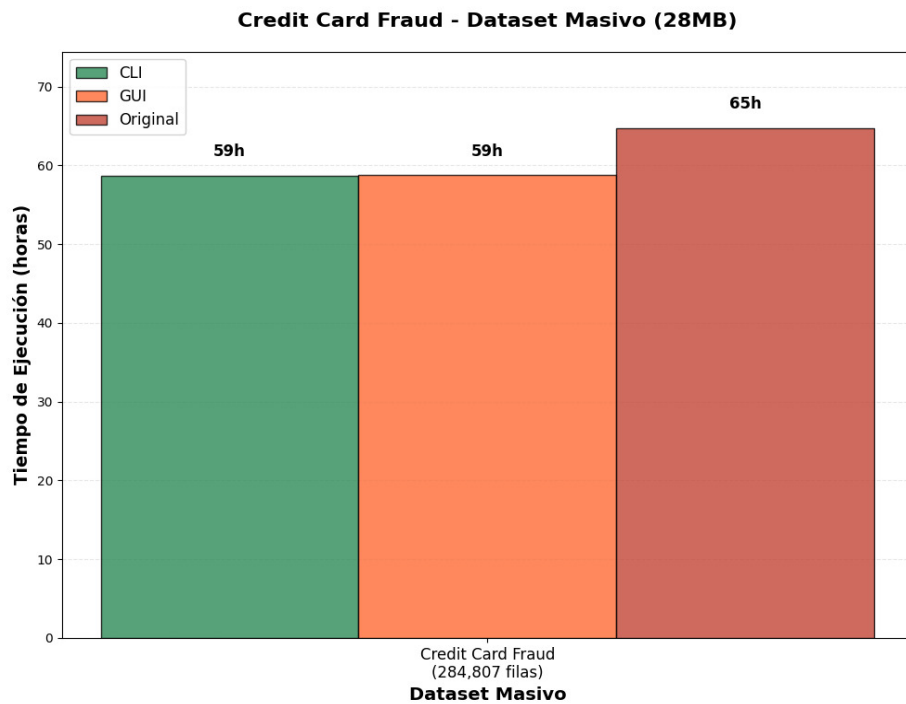


Fig 5.3 Experimentos tiempo imagen 2

Como se puede observar en estas imágenes, el tiempo de ejecución es notablemente más alto en el código antiguo (de media un 11% más lento comparado con la ejecución mediante interfaz gráfica) que no cuenta además con el tratamiento de datos, y con las mejoras obtenidas debido a las mejores estrategias de iteración que se han replicado para mantener unas condiciones comparables. Todos estos archivos se ejecutaron siguiendo el nivel de capilaridad balanceado.

El archivo de fraude de tarjetas de crédito tarda notablemente más tiempo en ejecutarse, se trata de un *dataset* masivo, cuyo estudio se ha hecho utilizando el nivel de capilaridad más alto disponible, que prueba 24 veces más combinaciones que el balanceado anterior.

Estas distintas pruebas nos dan un “insight” clave a nuestras decisiones arquitecturales. La diferencia de tiempo entre la ejecución a terminal y la ejecución con la interfaz gráfica mantiene una diferencia estable de entre 3 y 5 minutos por experimento, lo que nos indica una implementación correcta de archivos como medio de comunicación, ya que no atrasa significativamente unos procesos que, de por sí, son muy lentos.

Ahora centrándonos en el proceso ML encontramos otros gráficos. Para explicarlos ya se ha introducido en esta memoria el concepto de porque no se debe utilizar la *accuracy* como métrica global en sistemas de IA, ya que no distingue bien la capacidad de predecir clases minoritarias con mucho desbalance.

Para ello, se usan otras métricas como la F1-score o la *balanced-accuracy*, basadas en conceptos que tienen en cuenta la proporción de clases y su relación. Por ejemplo, la F1 utiliza estos conceptos:

- Precisión (*precision*): mide la proporción de predicciones positivas que son correctas. Es decir, de todos los casos que el modelo predijo como positivos, cuántos eran realmente positivos. Alta precisión indica pocos falsos positivos.
- Exhaustividad o sensibilidad (*recall*): mide la capacidad de recuperar positivos reales. De todos los casos que realmente son positivos, cuántos detecta el modelo. Alto recall indica pocos falsos negativos.

Como precisión y recall capturan facetas complementarias (evitar falsos positivos frente a evitar falsos negativos), la F1 score se utiliza para equilibrarlas en un único valor. La F1 es la media armónica de precisión y recall, por lo que solo es alta cuando ambas lo son; penaliza fuertemente los desequilibrios (por ejemplo, un modelo con precisión muy alta, pero recall bajo no tendrá una F1 alta).

Todas estas estadísticas se calculan a partir de comparar el resultado predicho por el clasificador, frente al real en el conjunto guardado para la validación (20% por defecto en el sistema), se pueden visualizar fácilmente gracias a la matriz de confusión.

La matriz de confusión proporciona la visión completa del comportamiento del clasificador. En binario, es una tabla 2×2 que cruza clases reales (filas) con clases predichas (columnas):

- Verdaderos positivos (TP): reales positivos clasificados como positivos.
- Falsos positivos (FP): reales negativos clasificados como positivos (alarmas injustificadas).
- Verdaderos negativos (TN): reales negativos clasificados como negativos.
- Falsos negativos (FN): reales positivos clasificados como negativos (casos omitidos).

Tras haber introducido estos conceptos, a continuación, se incluye un ejemplo que demuestra el funcionamiento del modelo.

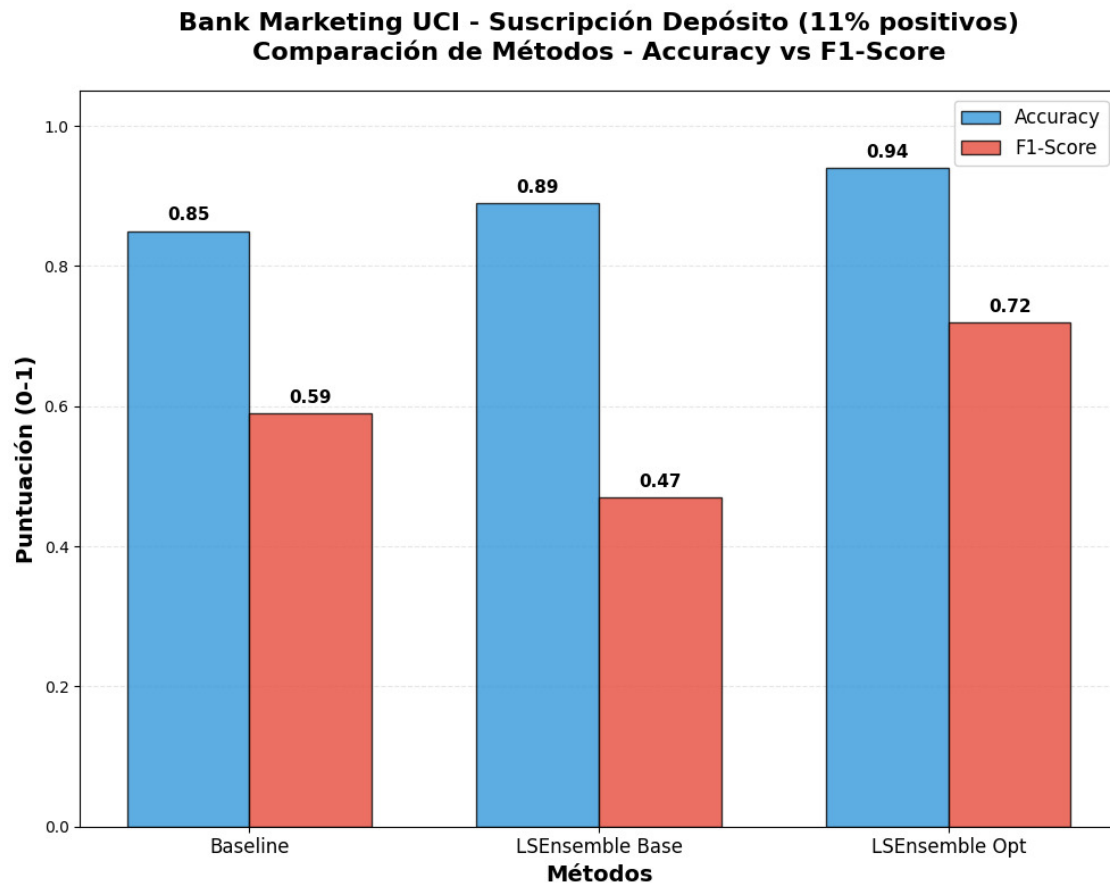


Fig 5.4 Comparación de métricas con modelos variados

En esta figura observamos un gráfico de barras que representa el valor de *accuracy* y *F1 score* utilizando diferentes modelos. El primer modelo utilizado llamado Baseline en la figura proviene de un artículo académico acerca del análisis de datos aplicado al marketing en bancos, y son las métricas extraídas del modelo con mejores prestaciones en la F-1 score (el modelo es un Naive Bayes) [19].

Se ha seleccionado para tener como referencia externa. Después, se ha entrenado el modelo LSEnsemble únicamente en la *Stage1* de nuestros parámetros (es decir, la mejor combinación de hiperparámetros ML) para este problema, y finalmente el modelo óptimo que encuentra la aplicación. Podemos ver una clarísima mejora con respecto al modelo base, debido a la mejora introducida por la combinación de SMOTE y el uso de Label Switching en este problema en particular.

Finalmente, observando la matriz de confusión de ambos modelos, se puede comprobar la verdadera mejora, que va mucho más allá de lo que nos dicen esas estadísticas:

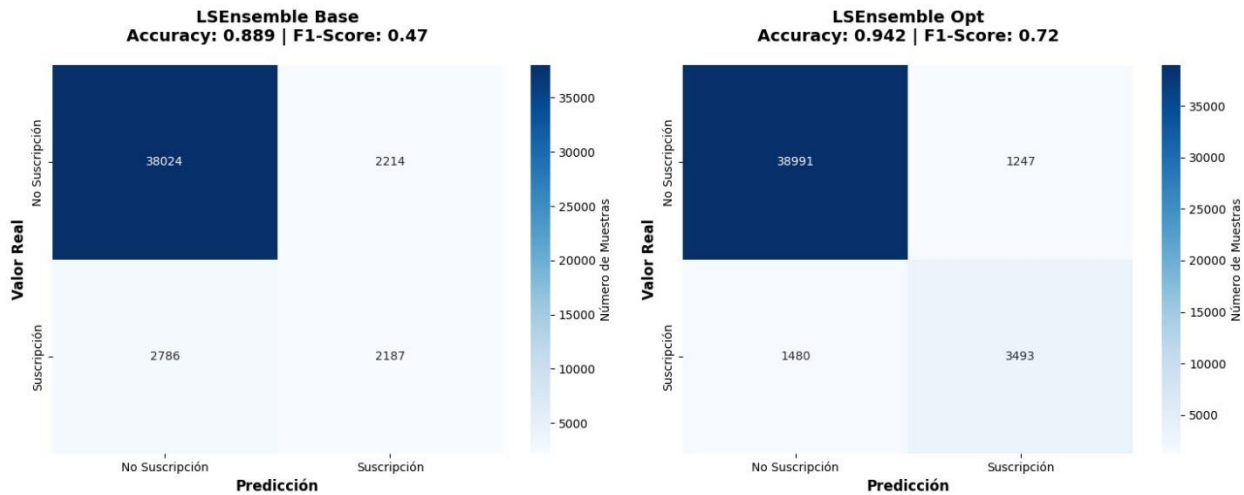


Fig 5.5 Comparación de matrices de confusión

Frente a *datasets* desbalanceados, los modelos tienen mucha facilidad de entender los verdaderos casos de la clase mayoritaria (verdaderos positivos, en las matrices esquina superior izquierda), y en esa categoría ambos modelos aciertan bastante. Es sin embargo en el resto de las categorías donde vemos brillar la optimización del modelo, y para ello faltaría añadir un apunte.

La optimización de los hiper-parámetros no es el único paso a la hora de analizar y montar un modelo predictivo, sino que forma parte de una cadena y, por lo tanto, estas estadísticas nos indican cómo debe funcionar, no obstante, hay factores que lo pueden mejorar que no forman parte del proceso de optimización.

Vemos una mejora clara en la detección de verdaderos negativos, pero la verdadera diferencia es combinada. A la izquierda vemos que el resto de las categorías reciben números muy parecidos de predicciones, y en la derecha los verdaderos negativos pasan a ser sobradamente más del doble de ambas categorías, indicándonos sin duda que el modelo está aprendiendo a diferenciarlos gracias a los cambios realizados.

5.2 Reportes.

Como resultado final del uso de la aplicación nos encontramos los reportes. Acorde a los requisitos y la especificación desarrolladas, deben ser simples, explicados a cualquier usuario con términos comprensibles a usuarios no técnicos, y se debe también desarrollar una previsualización para el mismo en la interfaz gráfica.

La previsualización se hace, tras transformar el código html gracias a la biblioteca `weasyprint`, pasándolo después a imágenes que Tkinter puede gestionar y mostrar.

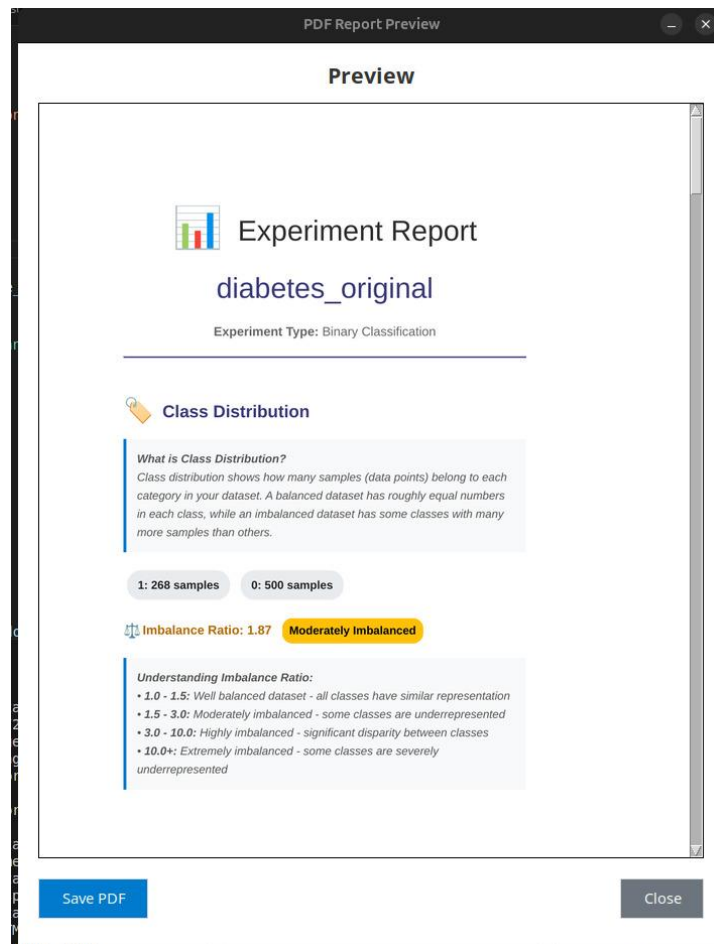


Fig 5.6 Previsualización PDFs – Página 1 reporte

Finalmente, estos son dos ejemplos de reportes. El primero es el reporte que se genera tras estudiar el csv de Diabetes Pima, que muestra la mejor estrategia para este dataset en concreto (ejecutando la app en inglés). Este primer reporte es distinto para experimentos multiclase, explica el tema de las dicotomías y las identifica, dando las mejores prestaciones para cada una. (Se escoge para enseñar el primero debido a la longitud extra del multiclase que no añade nueva información). Las imágenes comprenden de la segunda página en adelante ya que la primera aparece en la previsualización de la figura 5.6.

Los recortes de imágenes incluidos vienen de abrir el archivo PDF en un visualizador de navegador, para comprobar la correcta visualización del archivo en visores estándar.



Machine Learning Strategy

What is Ensemble Learning?

This experiment uses an ensemble approach where multiple neural network "experts" are trained together and their predictions are combined through voting. Each expert learns different aspects of the data, making the final model more robust and accurate than any single network.

Binary Classification Strategy

This experiment used a direct binary classification approach with an ensemble of neural network experts. The model was optimized specifically for this two-class problem using:

- **Label Switching Ensemble:** Multiple expert networks with label switching capabilities
- **Expert Voting System:** Each expert contributes to the final prediction through weighted voting
- **Advanced Regularization:** Dropout and other techniques to prevent overfitting
- **Custom Loss Functions:** Optimized for binary classification and class imbalance
- **Hyperparameter Optimization:** Systematic search for optimal network architecture and training parameters



Final Prediction

The ensemble combines predictions from all expert networks using sophisticated voting algorithms to make the final binary classification decision, resulting in more robust and accurate predictions.



Technical Implementation

Advanced Techniques Used:

The model employs cutting-edge machine learning techniques including ensemble methods, regularization, and custom optimization strategies.

Key Technical Features:

- **Expert Ensemble:** Multiple specialized neural networks working together
- **Dropout Regularization:** Prevents overfitting and improves generalization
- **Custom Loss Functions:** Optimized for handling imbalanced datasets
- **Batch Processing:** Efficient training with optimized batch sizes
- **Adaptive Learning:** Dynamic adjustment of learning parameters

4 / 8 | - 56% + | 🖼️ 🔍 🔗 ↶ ↷

 **Optimal Model Configuration**

Best Model Parameters Found:

This configuration represents the optimal values automatically determined during the optimization process. Each parameter has been fine-tuned to maximize performance on this specific dataset.

 **Neural Network Ensemble Configuration**

 **Core Architecture**

 **Number of Experts: 21**

Redes neuronales independientes que trabajan en conjunto. Configuración balanceada entre capacidad y eficiencia.

 **Hidden Size: 30 neurons**

Determina la capacidad de representación interna de cada experto. Más neurons = mayor capacidad pero también mayor riesgo de sobreajuste.

 **Activation Function: RELU**

ReLU: Función rápida y eficiente que evita el problema de gradientes que se desvanecen.

5 / 8 | - 75% + | 🖼️ 🔍 🔗 ↶ ↷

 **Training Configuration**

 **Batch Size: 128 samples**

Número de ejemplos procesados simultáneamente. Lotes más grandes = entrenamiento más estable pero más memoria.

 **Epochs: 50 iterations**

Número de veces que el modelo ve todo el dataset durante el entrenamiento.

 **Loss Function: CUSTOM**

Función personalizada optimizada para el desequilibrio de clases y Label Switching.

 **Dropout: 0**

Probabilidad de "apagar" neurons durante entrenamiento para prevenir sobreajuste. Sin dropout - el modelo confía en regularización por ensemble.

Label Switching Parameters

Alpha (α): 0.35

Controla el intercambio de etiquetas de clase mayoritaria $\rightarrow$ minoritaria. Valores más altos = más casos mayoritarios se convierten en minoritarios.

Beta (β): 0.1

Controla el intercambio de etiquetas de clase minoritaria $\rightarrow$ mayoritaria. Valores más bajos preservan mejor la clase minoritaria.

Q_RB_C: 5.0

Factor de refactor de coste. Ajusta la penalización por errores en diferentes clases para compensar el desequilibrio del dataset.

Q_RB_S: 2

Factor SMOTE (Synthetic Minority Oversampling). Controla la generación sintética de ejemplos de la clase minoritaria.

Data Distribution

Input Size: 8 features

Número de variables/features de entrada que recibe cada experto.

Data Mode: RANDOM

Modo aleatorio: Cada experto recibe una muestra aleatoria diferente de los datos.

RB per Expert: NO

Todos los expertos entrenan con el mismo conjunto de datos.

Optimizador: ADAM

ADAM: Optimizador adaptativo, eficiente para la mayoría de problemas.



Interpretation & Recommendations

Key Findings:

Moderate imbalance: Some classes are underrepresented. The model may perform better on majority classes.

What This Means:

- The model was trained using advanced techniques designed for binary classification
- An ensemble of expert neural networks was optimized for this two-class problem
- Hyperparameter optimization found the best configuration through systematic testing
- The final model combines predictions from multiple experts for robust classification
- Performance metrics indicate how well the model can make predictions on new, unseen data



Strategy Recommendations

Optimization Analysis:

Based on the performance across all dichotomies, here are the recommended configurations for future experiments and production deployment.

Strategy recommendations require completed experiments with performance metrics.

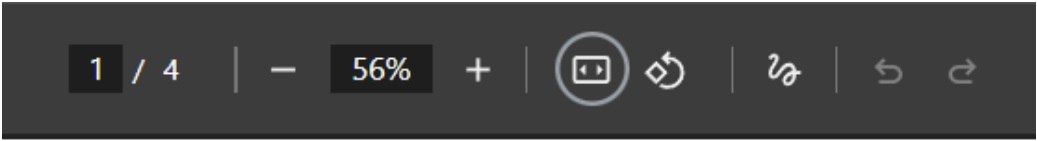
Generated by Guillermo Villar Sánchez • All Rights Reserved

Report created on September 07, 2025 at 10:14 PM

© 2025 UC3M

Fig 5.7 Reporte clase binaria Diabetes Pima

El otro tipo de reporte que genera nuestra aplicación es el de regeneración multiclase y estadísticas de rendimiento multiclase. Este explica en mayor detalle en que consiste las matrices ECOC, como se usan en este problema y que significan los resultados encontrados. En este caso es de la estrategia OVA con datos sintéticos.



Multiclass Model Reconstitution Report

Dataset Information

Dataset ID:	test_dataset
Strategy:	OVA
Number of Classes:	4
Class Names:	Class_A, Class_B, Class_C, Class_D
Test Size:	30.0%
Simulations:	10
Generated:	2025-09-07 22:41:22

Error-Correcting Output Codes (ECOC) Matrix

The OVA strategy decomposes the 4-class problem into 4 binary dichotomies using the following encoding matrix:

Class	Dichotomy 1	Dichotomy 2	Dichotomy 3	Dichotomy 4
Class_A	+1	+0	+0	+0
Class_B	+0	+1	+0	+0
Class_C	+0	+0	+1	+0
Class_D	+0	+0	+0	+1

Reconstitution Performance Summary

Metric	Average	Std. Deviation	95% Confidence Interval
Accuracy	0.8500	±0.0300	[0.7912, 0.9088]
Balanced Accuracy	0.8200	±0.0400	[0.7416, 0.8984]
Cohen's Kappa	0.7800	±0.0500	[0.6820, 0.8780]
Geometric Mean	0.8100	±0.0300	[0.7512, 0.8688]
Sensitivity	0.8000	±0.0400	[0.7216, 0.8784]

Performance Interpretation

Overall Performance: Good (85.0% accuracy)
Model Reliability: High (±3.0% variation)
Agreement Level: Good ($\kappa = 0.780$)

2 / 4

80%

Geometric Mean	0.8100	±0.0300	[0.7512, 0.8688]
Sensitivity	0.8000	±0.0400	[0.7216, 0.8784]

Performance Interpretation

Overall Performance: Good (85.0% accuracy)
Model Reliability: High (±3.0% variation)
Agreement Level: Good ($\kappa = 0.780$)

ECOC Effectiveness: The OVA strategy successfully decomposed the 4-class problem into 4 binary problems with effective error correction.

Detailed Simulation Results

Simulation	Accuracy	Balanced Acc.	Cohen's Kappa	Geometric Mean	Sensitivity
1	0.8300	0.8000	0.7500	0.7900	0.7800
2	0.8700	0.8400	0.8100	0.8300	0.8200
3	0.8400	0.8100	0.7600	0.8000	0.7900
4	0.8600	0.8300	0.7900	0.8200	0.8100
5	0.8500	0.8200	0.7800	0.8100	0.8000
6	0.8800	0.8500	0.8200	0.8400	0.8300
7	0.8200	0.7900	0.7400	0.7800	0.7700
8	0.8500	0.8200	0.7800	0.8100	0.8000
9	0.8600	0.8300	0.7900	0.8200	0.8100
10	0.8400	0.8100	0.7700	0.8000	0.7900

Detailed Model Reconstitution Process

1. Loading Experiment Configuration

The original multiclass experiment configuration is loaded. This includes the ECOC strategy (e.g., OVA, OVO), the number and names of classes, and the exact ECOC coding matrix that was used. This matrix is fundamental, as it defines how each binary classifier (dichotomy) contributes to the final decision.

2. Loading Original Dataset

The original CSV file containing the complete dataset with all classes is located and loaded. This serves as the starting point for simulating the training and validation process.

3. Loading Dichotomy Configurations

For each dichotomy (each column of the ECOC matrix), its optimal configuration is loaded from the hyperparameter optimization results. This file contains the model parameters (e.g., a Label Switching ensemble) that achieved the best performance for that specific binary classification task.

4. Simulation and Prediction

Multiple simulations are performed to ensure statistical robustness of the results. In each simulation:

4a. Data Splitting and Standardization

The original dataset is split into training and test sets (using 30% for testing). The data is standardized to have mean 0 and standard deviation 1, a standard practice for many Machine Learning models.

4b. Dichotomy Prediction

For each dichotomy, a model is instantiated with its optimal configuration and trained on the training data. Then, this model predicts the labels (-1 or 1) for the test set. The result is a matrix where each row is a test sample and each column is the prediction of a dichotomy.

5. Dichotomy Fusion (ECOC Decoding)

This is where the predictions from all binary classifiers are combined. For each test set sample:

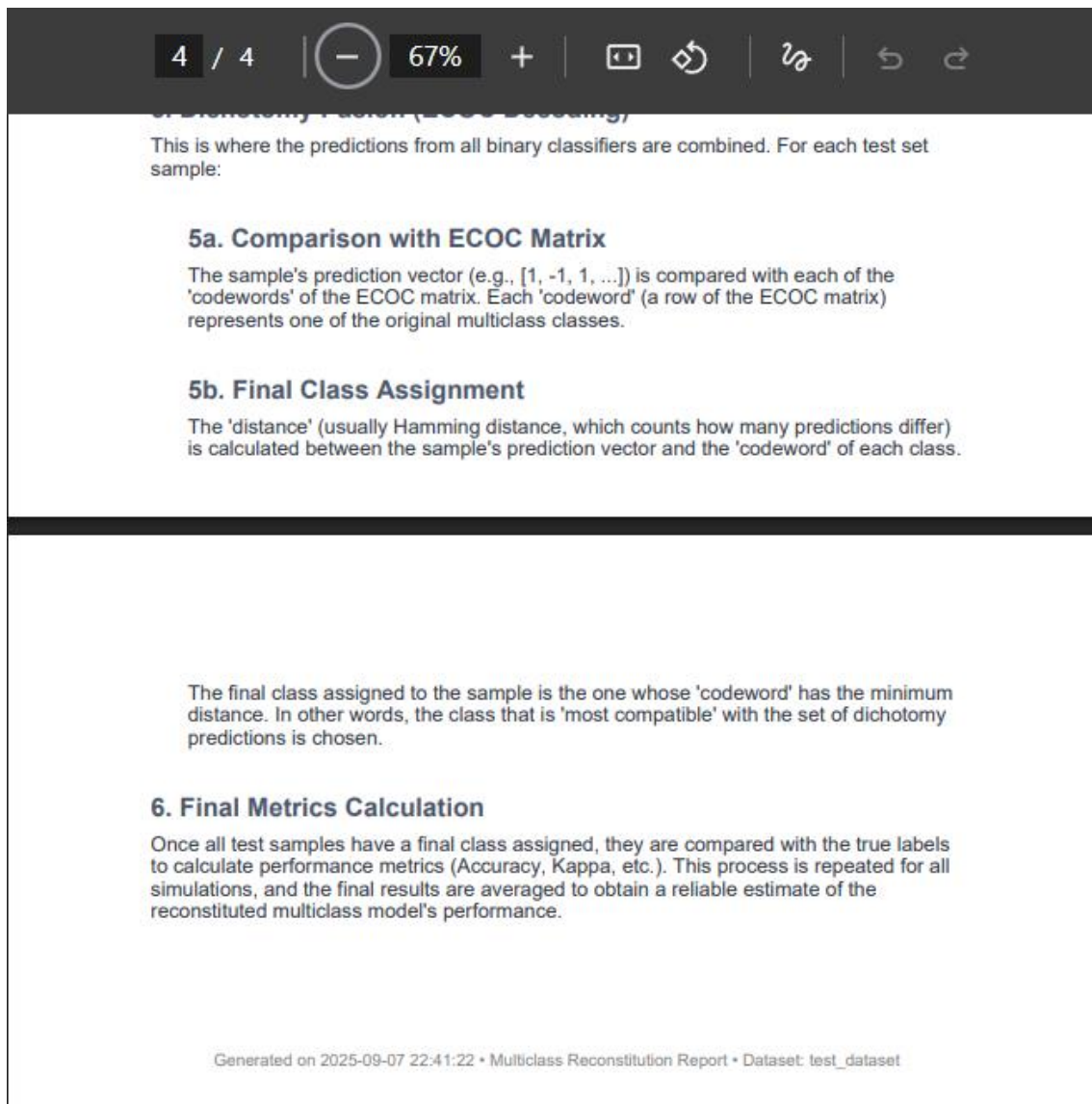


Fig 5.8 Reporte multiclase ECOC

6. CONCLUSIONES

6.1 Aprendizajes del desarrollo.

El desarrollo de este trabajo me ha beneficiado mucho como ingeniero informático, debido a que me ha enfrentado a los retos principales de un proyecto de software de forma estructurada.

Retos como partiendo de un software previo, no estructurado, plantear una especificación de requisitos e ir cumpliendo objetivos de manera estructurada y registrada de forma profesional en Git, esto no me ha llevado a descubrir retos distintos a los que me han ido surgiendo a lo largo de la titulación, pero si me ha obligado a enfrentarlos todos a la vez.

Además, he asentado y particularizado mi conocimiento en el área de IA, que es donde me quiero desarrollar, me ha permitido lidiar con las bases lógicas que cimientan esta tecnología y me ha dado una visión general que espero poder poner en práctica en el mundo laboral al que me enfrente.

En resumen, este proyecto ha supuesto una consolidación de los conocimientos adquiridos durante la carrera, pero también un paso más hacia la profesionalización: he aprendido a estructurar un desarrollo complejo, a integrar de forma coherente distintos conocimientos y tecnologías, y a gestionar de manera eficiente tanto el código como la documentación.

Considero que la experiencia me ha preparado no solo para abordar futuros retos técnicos, sino también para entender el valor de una metodología sólida, la importancia del trabajo bien documentado y la necesidad de mantener una mentalidad crítica y flexible frente a los problemas.

6.2 Mejoras futuras.

Aunque queda definido como requisito no cumplido, una gran mejora potencial para este proyecto (especialmente si se populariza entre sectores en particular) puede ser la funcionalidad descrita de conexión a un servicio web que disponga de los resultados ya ejecutados, de acuerdo con una configuración en particular, que los devuelva al usuario, mejorando la disponibilidad y contando, para ello, con una caché distribuida de resultados.

A su vez, se puede ampliar esta idea e incorporarle un apartado económico. Se podrían alquilar (obteniendo un beneficio) *clusters* de hardware para IA que ejecute muy rápidamente los experimentos y así poder informar al usuario antes.

Como ya se ha mencionado, la aplicación ha quedado restringida a términos locales. Esta decisión limita ciertas opciones de diseño, pero de la misma manera que la arquitectura de software va abstrayendo capas de iteración, se pueden iterar diversos modelos gracias a la mejora de cómputo de estos clústeres e incluso, ofrecer el servicio de entregar el mejor modelo, ya entrenado, al usuario.

7. MARCO REGULADOR

Dentro del mundo de la Inteligencia Artificial, se debe tener en cuenta que no es un ámbito exento de sesgos humanos y de responsabilidades de idéntica naturaleza. Como se aborda en este texto acerca del sesgo en la IA de la Chapman University, “los datos son la principal fuente de sesgo en los modelos de IA.” [\[20\]](#)

En este aspecto, el sistema desarrollado no va a introducir ningún sesgo, sino tratar de minimizar el sesgo que tienen los modelos de tender a la clase mayoritaria [\[2\]](#). Sin embargo, estos sesgos siguen presentes, los modelos siguen siendo “cajas negras” en cierto aspecto y las decisiones críticas deben seguir siendo tomadas por humanos expertos que puedan utilizar estos modelos como apoyo.

En lo relativo a la normativa, existen varios marcos normativos, leyes y licencias a las que ajustarse, tanto en España en particular como en el ámbito europeo.

El Reglamento UE 2016/679, de 27 de abril, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos, en adelante, RGPD, es la norma europea de protección de datos personales, que se aplica en España junto con la Ley Orgánica 3/2018, de 5 diciembre, de Protección de Datos Personales y Garantía de los Derechos Digitales (LOPDGDD).

Si bien este proyecto trata con bases de datos anónimas, y con las licencias pertinentes en su fase de desarrollo, está ideado para ser utilizado por personas con un conocimiento técnico limitado y, por ello, hay ciertos mecanismos en la entrada de datos que aseguran el cumplimiento de estas normativas.

Al recibir un nuevo *dataset*, se elimina cualquier información personal que identifique a una persona (emails, DNIs, nombres, etc.), ya que quedan fuera del campo de estudio de este proyecto.

El sistema no almacena información alguna sobre usuarios ni sobre los datos de un problema. Una vez encontrada la configuración óptima, aconseja acerca de la construcción de un modelo con el que tomar decisiones, pero la decisión final corresponde al usuario.

Sin embargo, y con relación al artículo 22 del RGPD que establece que toda persona tiene derecho a no ser objeto de decisiones basadas únicamente en tratamiento automatizado, se informará a los usuarios de esta normativa y otras relativas a los datos más sensibles.

Con respecto al Reglamento (UE) 2024/1689, de 13 de junio, de Inteligencia Artificial (AI Act) lo más significativo es la clasificación por niveles de riesgo de los sistemas de Inteligencia Artificial. Como ya se ha mencionado, la categorización del riesgo no se aplicaría al sistema como tal, sino a la utilización potencial del mismo.

En resumen, este proyecto ha sido diseñado bajo criterios que aseguran la conformidad con la normativa europea y española vigente en materia de protección de datos e inteligencia artificial. El sistema no solo evita el tratamiento de información personal innecesaria, sino que también promueve un uso responsable y explicable de modelos de IA. Así, se ofrece una herramienta que, sin comprometer la privacidad ni la legalidad,

permite a perfiles técnicos y no técnicos aplicar estrategias clasificatorias avanzadas de forma ética y segura.

En relación con los Objetivos de Desarrollo Sostenible (ODS), este proyecto puede contribuir de manera destacada a los siguientes:

ODS 9: Industria, innovación e infraestructura, al proporcionar herramientas de machine learning que facilitan la toma de decisiones más informadas y precisas en entornos de infraestructura e industria, promoviendo un uso eficiente de los recursos y la adopción de soluciones tecnológicas innovadoras.

ODS 16: Paz, justicia e instituciones sólidas, al garantizar que las decisiones apoyadas por los modelos de machine learning se tomen de manera segura y respetando la anonimización de los datos, protegiendo la privacidad de los usuarios y asegurando un marco de actuación transparente y confiable.

De esta manera, el proyecto combina innovación tecnológica con prácticas responsables y seguras, contribuyendo a la sostenibilidad ética y tecnológica en su ámbito de aplicación.

8. CONTEXTO E IMPACTO SOCIOECONOMICO

8.1 Presupuesto y fases.

El desarrollo del proyecto va a ser iterativo-incremental, se van creando nuevas funciones que cumplan con los requisitos, se guardan con git a través de un repositorio en Github, se validan y se construye la siguiente. Para su desarrollo, se acuerdan estas fases:

Fase 1 — Investigación y requisitos (diciembre-enero): revisión del estado del arte y definición de objetivos funcionales; alcance del MVC (GUI, modelos, conectores), criterios de estandarización de datasets y estrategia de generación de IDs.

Fase 2 — Diseño de arquitectura (enero-febrero): definición de interfaces entre GUI-conector-model, contratos de entrada/salida, estructura de directorios y convenciones de experimentos (runs, logs, resultados, informes).

Fase 3.1 — Ingesta y normalización de datos (febrero-marzo): pipeline de carga, detección de target, detección de multiclase, estandarización a formato interno y generación de identificadores de dataset.

Fase 3.2 — Conector (marzo-abril): orquestación entre GUI y model; llamadas a funciones del model para gestión de datasets y ejecución de experimentos; gestión de rutas y comunicación de dónde/cuándo leer/escribir.

Fase 3.3 — Modelos y ejecutores (abril-mayo): integración de modelos ML, scripts de ejecución, control de semillas, parametrización y almacenamiento de artefactos.

Fase 3.4 — Generación de informes (mayo-junio): cálculo de métricas, matrices de confusión, exportación a HTML/PDF, vista previa.

Fase 4 — Pruebas e integración (junio-julio): pruebas unitarias e integración de extremo a extremo, medidas de rendimiento y corrección de defectos.

Fase 5 — Refinamiento y hardening (julio): mejoras de UX, manejo de errores, trazabilidad, limpieza de código y documentación técnica.

Fase 6 — Redacción y cierre (Agosto-Septiembre): memoria, anexos, preparación de la defensa y empaquetado reproducible.

Para la visualización de estas fases se ha utilizado un diagrama de Gantt, para mantener las fases y su relación temporal acorde a las del desarrollo del proyecto.

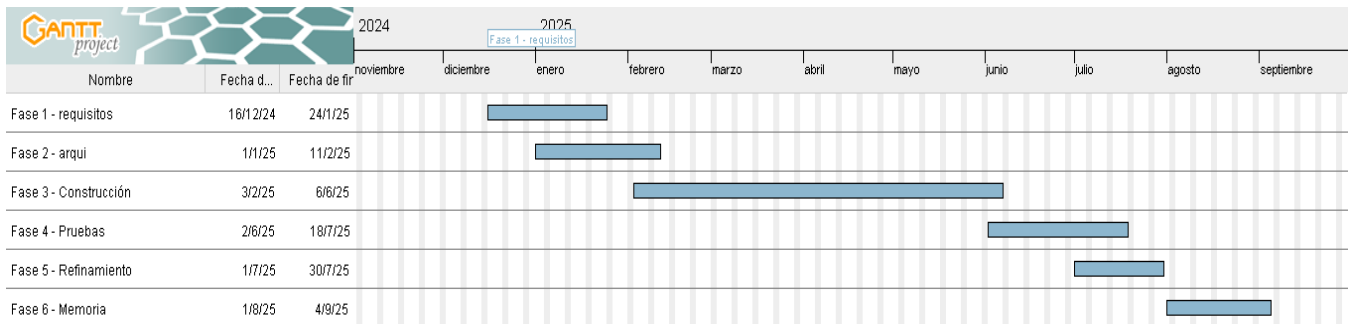


Fig 8.1 Diagrama de Gantt

Presupuesto del proyecto:

Costes directos

- Software: 0 € (Python, VS Code, scikit-learn, NumPy, Matplotlib, Jinja2, bibliotecas GUI; licencias libres o académicas).
- Cloud: 0 € (Google Colab académico disponible por cuenta universitaria).
- Asistentes de IA: 0 € uso de modelos de IA para apoyo puntual en diseño/plantillas de código, disponibles a través de cuenta universitaria.

Hardware y depreciación

- Equipo propio: portátil personal, Lenovo IdeaPad 15 16gb RAM Ryzen 7 5700U. Sin coste añadido debido a totalidad de ejecución local.

Coste de recursos humanos (valoración del trabajo)

- Perfil: Ingeniero/a junior de software/ML.
- Estimación de horas: 480 h (investigación, diseño, desarrollo, pruebas, documentación).
- Tarifa de referencia: 16 €/h (mercado junior).
- Cálculo: $480 \text{ h} \times 16 \text{ €/h} = 7680\text{€}$

Otros costes/contingencias

- Electricidad, periféricos, desplazamientos: marginales; se consideran gasto 0.

8.2 Impacto socioeconómico.

8.2.1.- Inteligencia Artificial

La inteligencia artificial ha dejado de ser una disciplina circunscrita al ámbito académico para convertirse en un motor de transformación económica y social a nivel mundial. Informes recientes de entidades como Goldman Sachs [21] o la consultora PwC estiman que esta tecnología podría impulsar el producto interior bruto (PIB) global entre un 7 % y un 14 % en la próxima década, como resultado del incremento de la eficiencia y la automatización en sectores clave.

Si bien muchas de estas proyecciones se centran en aplicaciones de inteligencia artificial generativa, el impacto de la IA va mucho más allá. Diversos análisis promovidos por organismos internacionales, como la Comisión Europea [22], destacan también el potencial de mejora de la productividad asociado al uso de modelos de IA en tareas estructuradas, como la clasificación, el análisis predictivo o la toma de decisiones basada

en datos. Es en este contexto donde se enmarca el presente trabajo, ya que el centro de muchos de los motores de la economía, presentes y futuros, tienen como base modelos clasificatorios.

8.2.2 - Modelos clasificatorios

Los modelos clasificatorios juegan un papel fundamental en la transformación digital de distintos sectores económicos. Además de automatizar distintas actividades humanas, posibilitan que profesionales de muchos sectores cuenten con una “segunda opinión” basada en los datos disponibles. Esto permite no sólo automatizar industrias, sino aumentar su eficacia y seguridad.

Entre los sectores que tienen una aplicación más relevante destaca el sector salud, en el cual modelos clasificatorios basados en imágenes de distintas pruebas médicas tienen el potencial de cambiar radicalmente la metodología y eficacia de los diagnósticos.

Esta revisión narrativa de la revista *Medicine (Baltimore)* [23] llega a la conclusión de que “la IA ha cambiado el paradigma de diagnósticos médicos incrementando la precisión, velocidad y armonización en múltiples áreas. Mediante la automatización de tareas repetitivas como detección de lesiones, segmentación de imágenes y análisis morfológico, la IA redujo el tiempo de diagnóstico en más de un 90% en algunos estudios y el volumen de datos en un 85%”.

Asimismo, en el sector financiero, los clasificadores se aplican en tareas como la detección de fraude, la evaluación de riesgos crediticios y el análisis de comportamiento de clientes. Se estima que la automatización de estos procesos mediante IA permitirá ahorrar hasta 440 mil millones de dólares anuales en el sector bancario global [24]. En la industria manufacturera, el uso de clasificadores para mantenimiento predictivo ha demostrado reducir entre un 18 % y un 25 % los costes operativos asociados a fallos de maquinaria.

8.2.3.- El problema del desbalance y sus consecuencias.

A pesar del potencial transformador de los modelos clasificatorios, uno de los retos más importantes que enfrentan es el desequilibrio en la distribución de clases en los datos de entrenamiento. Este fenómeno ocurre cuando una clase (por ejemplo, un caso positivo de enfermedad o un fraude bancario), se encuentra significativamente menos representada que otras. Como se ha documentado en diversos estudios, este desbalance puede generar sesgos importantes que afectan a la utilidad práctica del modelo [2].

Las consecuencias económicas de los errores inducidos por el desbalance son significativas: desde costes médicos adicionales por diagnósticos tardíos, hasta fraudes financieros no detectados o defectos de fabricación no identificados. De hecho, se estima que el fraude empresarial representa pérdidas globales de hasta 5,13 billones de dólares anuales [8], y muchos de estos casos no son detectados, precisamente por limitaciones en los clasificadores.

8.2.4.-Implicaciones regulatorias y sociales.

En sectores regulados, como el sanitario o el financiero, la fiabilidad de los modelos clasificatorios no solo es un requisito técnico, sino también normativo y ético. El uso de modelos con bajo desempeño en clases minoritarias puede comprometer la confianza de usuarios y reguladores, y frenar la adopción de herramientas con un gran potencial de desarrollo social. Esto es especialmente preocupante en un contexto en el que la legislación, como el RGPD y el AI Act, exige una gestión transparente y responsable de los sistemas de IA.

Por todo ello, garantizar la robustez de los modelos clasificatorios en situaciones de desbalance de clases no es sólo una cuestión técnica, sino también una necesidad económica, social y normativa. Este trabajo se inserta en ese marco, proponiendo una solución que permita a profesionales sin formación especializada evaluar la idoneidad de distintas estrategias ante problemas clasificatorios desbalanceados.

DECLARACIÓN USO IAG

He usado IAG en mi TFG

Marca lo que corresponda:

SI	NO
----	----

Si has marcado SI, completa las siguientes 3 partes de este documento:

Parte 1: declaración sobre comportamiento legal, ético y responsable

Ten presente que el uso de IAG conlleva unos riesgos y puede generar una serie de consecuencias académicas graves: el TFG no será evaluado por la Universidad si el uso de la IAG comporta la utilización de datos de carácter confidencial, materiales protegidos por derechos de autoría, o datos de carácter personal, y se hace sin cumplir las condiciones exigidas en cada caso (autorización de los interesados, autorización de los titulares, seguimiento de las instrucciones de la Universidad).

Pregunta	
1. En mi interacción con herramientas de IAG he facilitado datos de carácter confidencial contando siempre con la debida autorización de los interesados. La confidencialidad abarca toda información que una persona u organización desea proteger por razones legales, comerciales, de privacidad o estratégicas (como patentes o secretos comerciales).	
SÍ, he usado estos datos con la autorización de los interesados	NO, no he usado datos de carácter confidencial

2. En mi interacción con herramientas de IAG he facilitado materiales protegidos por derechos de autoría contando siempre con la autorización de los respectivos titulares.	
Sí, he usado estos materiales con autorización de los titulares de derechos de autor; o bien sin ella porque se ajustan a una de las excepciones o límites que permite la ley: • obra en dominio público • obra licenciada (licencias Creative Commons) • uso de fragmentos con fines de investigación (derecho de cita)	NO, no he usado materiales protegidos por derechos de autoría
3. En mi interacción con herramientas de IAG he facilitado datos de carácter personal con la debida autorización de los interesados.	
Sí, he usado estos datos con autorización de los interesados y conforme a las instrucciones contenidas en la guía aprobada por la Universidad	NO, no he usado datos de carácter personal
4. Mi utilización de la herramienta de IAG ha respetado sus términos de uso, así como los principios éticos esenciales, no orientándola de manera maliciosa a obtener un resultado inapropiado para el trabajo presentado, es decir, que produzca una impresión o conocimiento contrario a la realidad de los resultados obtenidos, que suplante mi propio trabajo o que pueda resultar en un perjuicio para las personas.	
SI	NO

Parte 2: declaración de uso técnico

Utiliza el siguiente modelo de declaración tantas veces como sea necesario, a fin de reflejar todos los tipos de iteración que has tenido con herramientas de IAG. Incluye un ejemplo por cada tipo de uso realizado donde se indique: *[Añade un ejemplo]*.

Declaro haber hecho uso del sistema de IAG (Nombre del sistema/herramienta IAG y versión: ChatGPT, Gemini, Copilot...) **para:**

Documentación y redacción:

- *Búsqueda de información o respuesta a preguntas concretas*

Declaro haber hecho uso del sistema de IA generativa Perplexity para buscar información y fuentes que explicasen bien el proceso de generación de dicotomías.

- *Búsqueda de bibliografía*

Declaro haber hecho uso del sistema de IA generativa Perplexity para búsqueda de información, en particular artículos científicos relativos al campo de estudio del TFG y búsqueda de documentación de herramientas tecnológicas utilizadas.

Desarrollar contenido específico

Se ha hecho uso de IAG como herramienta de soporte para el desarrollo del contenido específico del TFG, incluyendo:

- *Asistencia en el desarrollo de líneas de código (programación)*

Declaro haber hecho uso del sistema Github Copilot (con el modelo GPT 4.1) para el formato del archivo json de traducciones en mi proyecto, en concreto para escribir las llaves (que no el contenido) del mismo.

- *Asistencia en el desarrollo de líneas de código (programación)*

Declaro haber hecho uso del sistema Github Copilot (con el modelo GPT 4.1) para ayudarme a entender la embebición de CSS en código HTML para la generación de reportes.

- *Procesos de optimización*

Declaro haber hecho uso del sistema Github Copilot para resolver algunos errores en la previsualización de archivos PDF pasados a imágenes en Tkinter.

Parte 3: reflexión sobre utilidad

Aporta una valoración personal (formato libre) sobre las fortalezas y debilidades que has identificado en el uso de herramientas de IAG en el desarrollo de tu trabajo. Menciona si te ha servido en el proceso de aprendizaje, o en el desarrollo o en la extracción de conclusiones de tu trabajo.

Me parece que son herramientas que tienen mucho potencial en el ámbito del desarrollo de código, pero a su vez, tienen cierto peligro. Tienen mucho potencial porque me parece que la idea de poder hablar con tu estructura de archivos es muy potente, por ejemplo, cuando estás tratando de entender un código que no es el tuyo, pero esconde cierto peligro por la seguridad con la que cometen errores flagrantes que si nos revisan bien, pueden tener consecuencias graves.

Me ha servido en mi proceso de aprendizaje, me ha permitido ahorrar tiempo con tareas repetitivas y dedicarle energía a procesos de desarrollo de código con los que he enriquecido mis conocimientos.

9. BIBLIOGRAFÍA

- [1] Zawacki, B. Helba, G. Shih, J. Weber, J. Elliott, M. Combalia, N. Kurtansky, N. Codella, P. Culliton, and V. Rotemberg, 2020, "SIIM-ISIC Melanoma Classification," Kaggle. [En línea]. Disponible en: <https://www.kaggle.com/competitions/siim-isic-melanoma-classification>.
- [2] H. Ishwaran y R. O'Brien, "Commentary: The problem of class imbalance in biomedical data," *The Journal of Thoracic and Cardiovascular Surgery*, vol. 161, no. 6, pp. 1991–1992, Jun. 2021. Acceso: 5 de mayo 2025 [En línea]. Disponible en: [https://www.jtcvs.org/article/S0022-5223\(20\)31782-7/fulltext](https://www.jtcvs.org/article/S0022-5223(20)31782-7/fulltext)
- [3] S. Samoil, M. López-Cobo, E. Gómez, G. De Prato, F. Martínez-Plumed y B. Delipetrev, AI Watch. Defining Artificial Intelligence. Towards an operational definition and taxonomy of artificial intelligence. Luxemburgo: Oficina de Publicaciones de la Unión Europea, 2020. [En línea]. Disponible en: https://publications.jrc.ec.europa.eu/repository/bitstream/JRC118163/jrc118163_ai_watch_defining_artificial_intelligence_1.pdf
- [4] Salesforce, "Machine learning: Definición, Ventajas y Desventajas," Acceso: 8 de junio de 2025. [En línea]. Disponible en: <https://www.salesforce.com/es/resources/definition/machine-learning/>.
- [5] Universidad Complutense de Madrid, *¿Qué es el Machine Learning?*, Acceso: 8 de junio de 2025 [En línea]. Disponible en: <https://www.masterdatascienceucm.com/que-es-machine-learning/>.
- [6] I Belcic y C Stryker, "¿Qué es el aprendizaje supervisado?". IBM, Acceso: 8 de junio de 2025. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/supervised-learning>.
- [7] R Felip " Cómo escribir buenas historias de usuario", Apiumhub. Acceso: 8 de junio de 2025. [En línea]. Disponible en: <https://apiumhub.com/es/tech-blog-barcelona/como-escribir-buenas-historias-de-usuario/>
- [8] *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.
- [9] Mozilla Developer Network, *MVC*. Developer Docs. Acceso: 10 de julio de 2025. [En línea] Disponible en: <https://developer.mozilla.org/es/docs/Glossary/MVC>
- [10] N. Lewis, J. L. Bez, y S. Byna, "I/O in Machine Learning Applications on HPC Systems: A 360-degree Survey," *ACM Computing Surveys*. Acceso 14 de Julio de 2025. [En línea]. Disponible en: <https://arxiv.org/pdf/2404.10386>
- [11] B. German, 1987, "Glass Identification", UCI, doi: <https://archive.ics.uci.edu/dataset/42/glass+identification>

- [12] K. Nakai, 1996, "Ecoli", UCI, doi: <https://archive.ics.uci.edu/dataset/39/ecoli>
- [13] W. Wollberg, O. Mangasarian, N. Street y W. Street, 1995, "Breast Cancer Wisconsin (Diagnostic)", doi: <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>
- [14] UCI(machine learning) , "Pima Indians Diabetes Database", 1967, Kaggle, doi: <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>
- [15] M. Hopkins, E. Reeber, G. Fordman y J Suermondt, "Spambase", 1999, UCI, doi: <https://archive.ics.uci.edu/dataset/94/spambase>
- [16] M. Shahid, "Customer Churn Dataset", 2023, doi: <https://www.kaggle.com/datasets/muhammadshahidazeem/customer-churn-dataset>
- [17] Machine Learning Group ULB, "Credit Card Fraud Detection", 2018 , doi: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- [18] S. Moro, P. Rita, y P.Cortez, "Bank Marketing", 2012, doi: <https://archive.ics.uci.edu/dataset/222/bank+marketing>
- [19] S. Moro, P. Rita, y P.Cortez , "A data driven approach to predict the success of bank telemarketing", ScienceDirect. Acceso: 16 de Julio de 2025. [En línea]. Disponible en: <https://www.sciencedirect.com/science/article/abs/pii/S016792361400061X?via%3Dihub>
- [20] Chapman University. "Bias in AI" Champan University AI Hub. Acceso: 9 de Agosto de 2025. [En línea]. Disponible en: <https://www.chapman.edu/ai/bias-in-ai.aspx>
- [21] J. Hatzius, J. Briggs, D. Kodnani y G. Pierdomenico. "The Potentially Large Effects of Artificial Intelligence on Economic Growth". Goldman Sachs Global Economics Analyst. Acceso: 1 de Septiembre de 2025. [En línea]. Disponible en: <https://www.gspublishing.com/content/research/en/reports/2023/03/27/d64e052b-0f6e-45d7-967b-d7be35fabd16.html>
- [22] W. Simons, A. Turrini y L. Vivian, "Artificial Intelligence: Economic Impact, Opportunities, Challenges, Implications for Policy", Comisión Europea. Discussion Paper 210, 2024. [En línea]. Disponible en: https://economy-finance.ec.europa.eu/document/download/10867a27-f49f-4aa9-89bb-feb752513f78_en?filename=dp210_en_artificial+intelligence_0.pdf
- [23] V. Rotemberg et al., "AI in medical imaging: a narrative review," *Medicine (Baltimore)*, vol. 102, no. 6, p. e11813001, 2024. Acceso: 1 de

Septiembre 2025. [En línea]. Disponible en:
<https://pmc.ncbi.nlm.nih.gov/articles/PMC11813001/>

- [24] Instituto de Analistas Financieros, “El impacto económico de la inteligencia artificial en el sector bancario,” España, Informe, Papeles de la FEF num 64,2023. [En línea]. Disponible en: https://institutodeanalistas.com/wp-content/uploads/El-impacto-de-la-inteligencia-artificial-en-la-industria-financiera_.pdf

El repositorio del Proyecto es este: <https://github.com/Guillermo-villar/TFG>